



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

ASIGNATURA:

Aplicaciones Web

PROYECTO FIN DE CURSO ENTREGA 1B

**Módulo de Autenticación JWT y Acceso a Datos con Spring
Data JPA**

TEMA DEL PROYECTO:

**Sistema de Gestión de Pre-Sustentaciones de Trabajos de
Titulación**

Integrantes:

Álava Alvarado Jean Pierre
Moncayo Loor Xavier Alejandro
Zamora Arias Carla Esthefania

Docente:

Guerrero Ulloa Gleiston Ciceron

Repositorio GitHub:

<https://github.com/carla22072004/proyecto-fin-curso2026.git>

Periodo académico:

2025–2026

Quevedo – Ecuador

2026

Índice

Resumen ejecutivo	1
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos de la entrega	1
1.2.1. Objetivo general	1
1.2.2. Objetivos específicos	1
1.3. Alcance	2
1.4. Estructura del documento	2
2. Marco teórico	2
2.1. Arquitectura de aplicaciones web	2
2.2. Autenticación stateful y stateless	2
2.3. JSON Web Token	3
2.4. Spring Security	3
2.5. Mapeo objeto-relacional	3
2.6. Spring Data JPA	4
2.7. Hibernate	4
2.8. PostgreSQL	4
2.9. Migraciones de base de datos con Flyway	5
2.10. Redis y revocación de tokens	5
2.11. Protección de contraseñas con BCrypt	6
2.12. Seguridad de aplicaciones según OWASP	6
2.13. Control de acceso basado en roles	6
2.14. Configuración de CORS	7
2.15. Cabeceras HTTP de seguridad	7
2.16. Docker y Docker Compose	8
3. Diseño del sistema	8
3.1. Arquitectura general del sistema	8
3.2. Modelo C4 Nivel 2: contenedores	8
3.3. Diagrama de clases UML refactorizado	9
3.4. Correspondencia entre Java y PostgreSQL	11
3.4.1. Entidad Usuario	12
3.4.2. Entidad Solicitud	13
3.5. Diagrama Entidad-Relación	13
3.6. Diccionario de datos	14
3.6.1. Tabla usuario	14
3.6.2. Tabla solicitud	15
3.7. Diagrama de secuencia de autenticación JWT	15
3.8. Migración inicial de base de datos con Flyway	16
4. Implementación	17
4.1. Estructura del repositorio	17
4.2. Tecnologías utilizadas	20
4.3. Configuración del backend	20
4.4. Implementación del inicio de sesión	21
4.5. Generación del token JWT	22
4.6. Implementación del filtro JWT	22
4.7. Implementación del acceso a datos	23
4.8. Endpoints principales implementados	24
4.9. Pruebas de integración con Postman	25
4.9.1. Prueba de inicio de sesión	25
4.9.2. Consulta de solicitudes con JWT	25
4.10. Implementación del frontend Angular	26
4.10.1. Servicio de autenticación	26
4.10.2. Interceptor JWT	26

4.10.3. Protección de rutas con AuthGuard	27
4.10.4. Interfaz de inicio de sesión	27
5. Pruebas	27
5.1. Casos de prueba de autenticación	27
5.2. Resultados de las pruebas manuales	28
5.3. Pruebas automatizadas propuestas	28
5.4. Métricas de rendimiento	29
6. Seguridad implementada	29
6.1. Evidencias de seguridad	30
6.2. Aspectos de seguridad pendientes	31
7. Docker Compose y despliegue	31
7.1. Instrucciones de ejecución	32
8. Resultados obtenidos	32
9. Conclusiones	33
9.1. Conclusión del primer objetivo	33
9.2. Conclusión del segundo objetivo	33
9.3. Conclusión del tercer objetivo	33
9.4. Conclusión del cuarto objetivo	33
Referencias bibliográficas	34

Resumen ejecutivo

El presente proyecto desarrolló el primer módulo funcional del Sistema de Gestión de Pre-Sustentaciones de Trabajos de Titulación, orientado a centralizar y mejorar los procesos académicos relacionados con el registro de solicitudes, la gestión de usuarios y el seguimiento de las actividades previas a la sustentación. El sistema respondió a la necesidad de reducir la dispersión de información, los retrasos administrativos y la falta de trazabilidad ocasionada por el uso de procesos manuales. Para su construcción se utilizó una arquitectura cliente-servidor compuesta por un frontend desarrollado con Angular y un backend implementado con Java y Spring Boot. La autenticación se gestionó mediante Spring Security y tokens JSON Web Token, permitiendo proteger los endpoints y controlar el acceso de los usuarios según sus credenciales. El acceso a la información se realizó con Spring Data JPA e Hibernate sobre PostgreSQL, utilizando entidades, repositorios, servicios y controladores organizados por capas. Como resultado, se obtuvo una aplicación capaz de autenticar usuarios, generar tokens de acceso, procesar solicitudes HTTP protegidas y ejecutar operaciones de consulta y gestión sobre las entidades principales del sistema. Además, se documentó la arquitectura mediante diagramas UML, el modelo de datos y los controles de seguridad aplicados. La integración de Redis, Flyway, pruebas automatizadas y Docker Compose se estableció como parte de la ampliación técnica prevista para fortalecer la seguridad, reproducibilidad y despliegue del sistema.

1. Introducción

1.1. Contexto y motivación

La gestión de las pre-sustentaciones de trabajos de titulación comprende actividades como el registro de solicitudes, la revisión de documentos, la asignación de docentes, la programación de fechas y el seguimiento de los estados de cada proceso. Cuando estas tareas se realizan mediante correos electrónicos, archivos independientes o registros manuales, pueden presentarse retrasos, duplicidad de información, pérdida de documentos y dificultades para conocer el estado actualizado de una solicitud.

Ante esta problemática, se planteó el desarrollo de un Sistema de Gestión de Pre-Sustentaciones de Trabajos de Titulación que centralice la información académica y permita que estudiantes, docentes y personal administrativo accedan a las funciones correspondientes según su rol. El sistema fue desarrollado mediante una arquitectura cliente-servidor, utilizando Angular para la interfaz web y Spring Boot para la implementación de la API REST.

En esta entrega se trabajó principalmente en el módulo de autenticación mediante JSON Web Token y en el acceso a datos con Spring Data JPA e Hibernate. Estos componentes constituyen la base para proteger los recursos del sistema, identificar a los usuarios y gestionar la información almacenada en PostgreSQL.

1.2. Objetivos de la entrega

1.2.1. Objetivo general

Implementar y documentar el primer módulo funcional del Sistema de Gestión de Pre-Sustentaciones de Trabajos de Titulación, incorporando autenticación basada en JWT y acceso a datos mediante Spring Data JPA sobre PostgreSQL.

1.2.2. Objetivos específicos

1. Implementar el proceso de autenticación de usuarios mediante Spring Security y tokens JWT.
2. Organizar el backend mediante entidades, repositorios, servicios y controladores REST.
3. Gestionar el acceso y almacenamiento de información utilizando Spring Data JPA, Hibernate y PostgreSQL.
4. Documentar la arquitectura, las clases principales, los endpoints, los controles de seguridad y las pruebas realizadas sobre el sistema.

1.3. Alcance

La presente entrega comprendió la documentación e implementación de los componentes relacionados con el inicio de sesión, el registro y la gestión de usuarios, la generación y validación de tokens JWT, la protección de endpoints y la comunicación entre el backend y la base de datos PostgreSQL.

También se incluyó la descripción de la estructura del proyecto, las entidades implementadas, los repositorios JPA, los servicios, los controladores REST y la integración con el frontend Angular. Como entidad principal del sistema se consideró la solicitud de pre-sustentación, debido a que representa el punto de inicio del proceso académico gestionado por la aplicación.

Los componentes relacionados con la revocación de tokens mediante Redis, migraciones completas con Flyway, pruebas automatizadas, métricas de rendimiento y despliegue mediante Docker Compose se documentaron como elementos complementarios y de ampliación técnica para las siguientes etapas del proyecto.

1.4. Estructura del documento

El documento se encuentra organizado en secciones que describen el fundamento teórico, el diseño del sistema, la arquitectura tecnológica, el modelo de clases, el modelo de datos y el flujo de autenticación JWT. Posteriormente, se presenta la implementación del backend y del frontend, los endpoints disponibles, las pruebas realizadas, los controles de seguridad y la propuesta de despliegue. Finalmente, se incluyen los resultados obtenidos, las conclusiones y las referencias bibliográficas.

2. Marco teórico

2.1. Arquitectura de aplicaciones web

Una aplicación web moderna se encuentra formada por diferentes componentes que colaboran mediante una arquitectura cliente-servidor. El cliente corresponde a la interfaz con la que interactúa el usuario, mientras que el servidor procesa las solicitudes, aplica las reglas del negocio y administra el acceso a la información almacenada en la base de datos.

En el sistema desarrollado, Angular cumple la función de cliente web y Spring Boot funciona como servidor de aplicaciones. La comunicación entre ambos componentes se realiza mediante una API REST, utilizando el protocolo HTTP y representaciones de datos en formato JSON. Esta separación permite desarrollar y mantener de forma independiente la interfaz, la lógica del negocio y la persistencia de datos.

La arquitectura implementada también facilita la protección de los recursos del sistema, debido a que las solicitudes enviadas desde Angular deben ser evaluadas por Spring Security antes de acceder a los controladores y servicios del backend.

2.2. Autenticación stateful y stateless

La autenticación permite verificar la identidad de un usuario antes de concederle acceso a los recursos de una aplicación. En un modelo *stateful*, el servidor mantiene una sesión asociada con cada usuario autenticado. Generalmente, el cliente recibe un identificador de sesión que debe enviar en las solicitudes posteriores.

En el modelo *stateless*, el servidor no conserva una sesión tradicional para cada usuario. En su lugar, después de validar las credenciales, emite un token que contiene la información necesaria para identificar al usuario. El cliente debe enviar este token en cada solicitud dirigida a un recurso protegido.

El enfoque stateless resulta apropiado para aplicaciones de página única o SPA, debido a que permite separar el frontend del backend y facilita el consumo de la misma API desde diferentes clientes. Sin embargo, requiere aplicar controles sobre la firma, expiración, almacenamiento y revocación de los tokens para evitar accesos no autorizados.

En este proyecto se utilizó una política de sesión stateless mediante Spring Security. Después del inicio de sesión, el backend genera un token JWT que debe enviarse en el encabezado **Authorization** de las solicitudes protegidas.

2.3. JSON Web Token

JSON Web Token, conocido como JWT, es un estándar abierto definido en el RFC 7519 para representar declaraciones o *claims* entre dos partes mediante un formato compacto y seguro para URL [1]. Su

uso es frecuente en sistemas de autenticación y autorización de APIs REST.

Un token JWT se encuentra compuesto por tres partes separadas mediante puntos:

1. **Header:** contiene información sobre el tipo de token y el algoritmo utilizado para firmarlo.
2. **Payload:** contiene las declaraciones relacionadas con el usuario, como su identificador, correo, rol, fecha de emisión y fecha de expiración.
3. **Signature:** permite verificar que el contenido del token no fue modificado después de su emisión.

La estructura general de un token JWT es la siguiente:

`header.payload.signature`

En el sistema, el token fue firmado mediante un secreto compartido y enviado al frontend después de validar las credenciales del usuario. En cada solicitud protegida, Angular agrega el token al encabezado HTTP mediante el siguiente formato:

`Authorization: Bearer <token>`

La utilización de un token firmado permite al backend comprobar su integridad antes de autorizar el acceso. No obstante, la información del *payload* no debe considerarse cifrada, debido a que puede ser decodificada. Por esta razón, no deben almacenarse contraseñas ni información sensible dentro del JWT.

2.4. Spring Security

Spring Security es el componente del ecosistema Spring utilizado para implementar mecanismos de autenticación, autorización y protección de aplicaciones Java. Su arquitectura se basa en una cadena de filtros que intercepta las solicitudes HTTP antes de que estas alcancen los controladores del sistema [2].

Uno de sus componentes principales es `SecurityFilterChain`, que permite configurar los endpoints públicos, los recursos protegidos, la política de sesiones, los controles CORS y los filtros personalizados. El componente `AuthenticationManager` coordina el proceso de autenticación y delega la validación de las credenciales al proveedor configurado.

La interfaz `UserDetailsService` permite recuperar los datos del usuario desde una fuente de almacenamiento. En este proyecto, su implementación consulta el repositorio de usuarios y obtiene la información registrada en PostgreSQL. Posteriormente, Spring Security compara la contraseña recibida con el hash almacenado.

Cuando la autenticación es válida, la información del usuario se registra temporalmente en el `SecurityContextHolder`. De esta manera, los controladores y servicios pueden identificar al usuario autenticado y comprobar sus permisos durante la solicitud.

El proyecto también utiliza un filtro JWT que intercepta las peticiones, extrae el token del encabezado `Authorization`, verifica su validez y establece la autenticación correspondiente dentro del contexto de seguridad.

2.5. Mapeo objeto-relacional

El mapeo objeto-relacional, conocido como ORM por sus siglas en inglés, es una técnica que permite relacionar las clases y objetos de una aplicación con las tablas de una base de datos relacional. Su propósito es disminuir las diferencias existentes entre el modelo orientado a objetos utilizado en Java y el modelo tabular utilizado por los sistemas gestores de bases de datos [4].

Mediante un ORM, una clase Java puede representar una tabla, mientras que sus atributos representan las columnas correspondientes. De esta forma, el desarrollador puede gestionar la información mediante objetos sin tener que construir manualmente todas las instrucciones SQL requeridas para insertar, consultar, actualizar o eliminar registros.

En comparación con el uso directo de JDBC, el enfoque ORM reduce la cantidad de código repetitivo relacionado con conexiones, preparación de consultas y transformación de resultados. También facilita la definición de relaciones entre entidades, la gestión de transacciones y la validación de los datos antes de almacenarlos.

En el proyecto, las clases del dominio fueron definidas como entidades mediante la anotación `@Entity`. Las claves primarias se identificaron con `@Id`, mientras que las relaciones entre las entidades se representaron utilizando anotaciones como `@OneToMany`, `@ManyToOne` y `@OneToOne`.

2.6. Spring Data JPA

Spring Data JPA es un módulo del ecosistema Spring que proporciona soporte para la persistencia de datos mediante Jakarta Persistence API. Su objetivo es reducir el código repetitivo necesario para implementar la capa de acceso a datos [3].

La abstracción de repositorios permite declarar interfaces asociadas con una entidad y su tipo de identificador. Por ejemplo, una interfaz puede extender `JpaRepository<Solicitud, Long>` para obtener automáticamente operaciones de creación, consulta, actualización, eliminación, paginación y ordenamiento.

Una estructura básica de repositorio se representa de la siguiente manera:

```
public interface SolicitudRepository
    extends JpaRepository<Solicitud, Long> {
}
```

Además de las operaciones heredadas, Spring Data JPA permite declarar consultas derivadas a partir del nombre de los métodos. Por ejemplo:

```
Optional<Usuario> findByEmail(String email);
List<Solicitud> findByEstado(String estado);
```

En el sistema de gestión de pre-sustentaciones, los repositorios permiten consultar y almacenar usuarios, estudiantes, docentes, solicitudes, documentos, tribunales y evaluaciones. Esta organización evita que los controladores accedan directamente a la base de datos y mantiene una separación adecuada entre las capas de la aplicación.

2.7. Hibernate

Hibernate es una implementación de mapeo objeto-relacional para aplicaciones Java. En una aplicación Spring Boot, funciona como proveedor de persistencia y se encarga de transformar las operaciones realizadas sobre entidades Java en instrucciones compatibles con la base de datos relacional [4].

Hibernate administra el ciclo de vida de las entidades mediante estados como transitorio, persistente y separado. También permite gestionar relaciones entre objetos, ejecutar consultas y sincronizar los cambios realizados en memoria con la información almacenada en la base de datos.

En el proyecto, Hibernate interpreta las anotaciones JPA presentes en las entidades y realiza el mapeo entre los tipos Java y los tipos PostgreSQL. Por ejemplo, un atributo de tipo `Long` puede relacionarse con una columna `BIGINT`, mientras que un atributo `String` puede almacenarse como `VARCHAR`.

El uso de Hibernate junto con Spring Data JPA permitió desarrollar la capa de persistencia sin concatenar instrucciones SQL dentro del código Java. Esta decisión también ayuda a disminuir el riesgo de inyección SQL cuando las consultas se realizan mediante repositorios y parámetros administrados por el framework.

2.8. PostgreSQL

PostgreSQL es un sistema gestor de bases de datos relacional utilizado para almacenar de forma estructurada la información del sistema. Proporciona tipos de datos, restricciones, índices, claves primarias, claves foráneas y mecanismos transaccionales necesarios para conservar la integridad de los registros [5].

En el proyecto, PostgreSQL almacena la información de los usuarios y de las entidades relacionadas con el proceso de pre-sustentación. Entre ellas se encuentran las solicitudes, los estudiantes, los docentes, los anteproyectos, los tribunales, las fechas programadas y las evaluaciones.

La definición del esquema utiliza restricciones como `NOT NULL`, `UNIQUE`, `CHECK` y `FOREIGN KEY`. Estas restricciones complementan las validaciones realizadas en el backend y evitan que la base de datos acepte información que no cumpla las reglas establecidas.

PostgreSQL también permite utilizar tipos como `BIGSERIAL` para identificadores autogenerados, `BOOLEAN` para valores lógicos y `TIMESTAMPTZ` para registrar fechas y horas considerando la zona temporal.

2.9. Migraciones de base de datos con Flyway

Flyway es una herramienta utilizada para controlar y versionar los cambios realizados sobre el esquema de una base de datos. Las migraciones se almacenan como archivos dentro del repositorio y se ejecutan de forma ordenada cuando la aplicación inicia o cuando se utiliza el comando de migración correspondiente [6].

Las migraciones versionadas utilizan nombres como:

```
V1__schema_inicial.sql
V2__agregar_tabla_solitudes.sql
V3__agregar_indices.sql
```

Cada migración versionada se ejecuta una sola vez y su aplicación queda registrada en la tabla de historial de Flyway. Por esta razón, una migración que ya fue ejecutada no debe modificarse; los nuevos cambios deben agregarse en un archivo con una versión superior.

Flyway también admite migraciones repetibles, identificadas mediante el prefijo `R__`. Estas migraciones se ejecutan nuevamente cuando cambia su contenido y se utilizan principalmente para vistas, procedimientos, funciones o datos de referencia.

Un ejemplo de migración repetible es:

```
R__crear_vistas_reportes.sql
```

Para la presente entrega se planteó el archivo `V1__schema_inicial.sql`, encargado de establecer las tablas principales, las restricciones, los índices únicos y el trigger de actualización automática del campo `actualizado_en`. La incorporación completa de estas migraciones se consideró parte del fortalecimiento de la reproducibilidad del proyecto.

2.10. Redis y revocación de tokens

Redis es un sistema de almacenamiento de datos en memoria basado en estructuras clave-valor. Se caracteriza por proporcionar operaciones de lectura y escritura de baja latencia y por permitir que las claves tengan un tiempo de vida determinado [7].

En una arquitectura basada en JWT, el servidor no mantiene una sesión tradicional del usuario. Como consecuencia, un token válido puede continuarse utilizándose hasta que alcance su fecha de expiración, incluso después de que el usuario haya cerrado sesión. Para controlar esta situación, se puede implementar una lista de revocación o *blacklist*.

La estrategia consiste en almacenar en Redis el identificador único del token, denominado *JWT ID* o `jti`, cuando el usuario ejecuta el cierre de sesión. Antes de autorizar una solicitud, el filtro JWT consulta si el identificador está registrado en la lista de revocación.

Cada registro debe almacenarse con un tiempo de vida equivalente al tiempo restante de validez del token. Una vez transcurrido este periodo, Redis elimina automáticamente la clave, debido a que el token ya no puede utilizarse.

La estructura conceptual de una entrada de revocación es la siguiente:

```
Clave: blacklist:<jti>
Valor: revoked
TTL: tiempo restante del token
```

En la arquitectura propuesta para el sistema, Redis se consideró como el componente encargado de administrar la revocación de tokens y apoyar futuras funciones de almacenamiento temporal y caché.

2.11. Protección de contraseñas con BCrypt

Las contraseñas no deben almacenarse directamente en texto plano ni mediante algoritmos de cifrado reversible. En su lugar, deben procesarse mediante una función de hash especializada para contraseñas, acompañada de un factor de trabajo que aumente el costo computacional de los intentos de adivinación [8].

BCrypt es una función utilizada para transformar una contraseña en una representación no reversible. Cada hash incorpora una sal aleatoria, por lo que dos usuarios con la misma contraseña pueden obtener resultados diferentes.

La recomendación de OWASP establece un factor de trabajo mínimo de 10 para BCrypt. En los requerimientos de esta entrega se estableció un costo de 12, que puede configurarse en Spring Security mediante:

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}
```

Durante el registro, la contraseña ingresada debe procesarse antes de almacenar al usuario. Durante el inicio de sesión, Spring Security compara la contraseña recibida con el hash mediante el método `matches()`, sin recuperar ni descifrar la contraseña original.

El atributo que contiene el hash tampoco debe incluirse en las respuestas JSON del sistema. Para evitar su exposición, puede utilizarse un DTO de respuesta o la anotación `@JsonIgnore` sobre el campo correspondiente.

2.12. Seguridad de aplicaciones según OWASP

El proyecto OWASP Top 10 presenta una clasificación de riesgos relevantes para las aplicaciones web. Para la presente entrega se consideraron principalmente las categorías de pérdida de control de acceso, fallas criptográficas, inyección, configuración de seguridad incorrecta y fallas de identificación y autenticación [9].

Los controles relacionados con estas categorías se describen a continuación:

- **A01 – Pérdida de control de acceso:** se controla mediante roles, autorización de endpoints y anotaciones como `@PreAuthorize`.
- **A02 – Fallas criptográficas:** se reduce utilizando BCrypt para proteger las contraseñas y claves suficientemente extensas para firmar los tokens.
- **A03 – Inyección:** se disminuye mediante Spring Data JPA, consultas parametrizadas y la eliminación de concatenaciones SQL.
- **A05 – Configuración de seguridad incorrecta:** se atiende configurando CORS, cabeceras HTTP, variables de entorno y restricciones explícitas de acceso.
- **A07 – Fallas de identificación y autenticación:** se controla mediante validación de credenciales, expiración de tokens, protección de contraseñas y revocación de tokens.

La seguridad no depende de un único componente. Es necesario aplicar controles en la interfaz, el backend, la base de datos, la infraestructura y el proceso de despliegue.

2.13. Control de acceso basado en roles

El control de acceso basado en roles permite asignar permisos según las responsabilidades de cada usuario dentro del sistema. En Spring Security, los roles pueden validarse tanto en la configuración de las rutas como directamente sobre los métodos.

Un ejemplo de protección de un método administrativo es:

```
@PreAuthorize("hasRole('ADMIN')")
public void eliminarUsuario(Long id) {
    // Logica de negocio
}
```

En el Sistema de Gestión de Pre-Sustentaciones, los permisos permiten diferenciar las acciones disponibles para estudiantes, docentes, miembros del tribunal y personal administrativo. De esta forma, un estudiante puede registrar o consultar su solicitud, mientras que las operaciones de revisión, programación y administración corresponden a usuarios con funciones específicas.

Este mecanismo se relaciona con la prevención de accesos no autorizados, debido a que el usuario debe encontrarse autenticado y poseer el rol requerido antes de ejecutar una operación protegida.

2.14. Configuración de CORS

Cross-Origin Resource Sharing, conocido como CORS, es un mecanismo que permite determinar qué orígenes pueden realizar solicitudes hacia una aplicación web. Su configuración es necesaria cuando el frontend Angular y el backend Spring Boot se ejecutan desde dominios o puertos diferentes [10].

Una configuración explícita debe establecer:

- Los orígenes autorizados.
- Los métodos HTTP permitidos.
- Los encabezados aceptados.
- La utilización de credenciales cuando sea necesaria.
- El tiempo durante el cual el navegador puede conservar la respuesta de prevalidación.

No se recomienda permitir indiscriminadamente todos los orígenes en un entorno de producción. En su lugar, debe declararse la dirección exacta desde la que se sirve el frontend.

Una configuración conceptual para desarrollo puede permitir:

```
http://localhost:4200
```

Mientras que en producción debe utilizarse únicamente el dominio autorizado para la aplicación.

2.15. Cabeceras HTTP de seguridad

Las cabeceras HTTP de seguridad proporcionan instrucciones al navegador para limitar determinados comportamientos que pueden ser aprovechados en ataques. Spring Security permite configurar varias de estas cabeceras dentro de la cadena de filtros [11].

Entre las cabeceras requeridas para la entrega se encuentran:

- **X-Content-Type-Options: nosniff**, que evita que el navegador intente interpretar un recurso con un tipo diferente al declarado.
- **X-Frame-Options: DENY**, que impide que la aplicación sea mostrada dentro de marcos de otros sitios y ayuda a reducir ataques de *clickjacking*.
- **Content-Security-Policy**, que establece los orígenes permitidos para cargar scripts, estilos, imágenes y otros recursos.

Una política inicial restrictiva puede expresarse de la siguiente forma:

```
Content-Security-Policy: default-src 'self'
```

La política definitiva debe adaptarse a los recursos que utiliza el frontend, evitando habilitar orígenes innecesarios.

2.16. Docker y Docker Compose

Docker permite empaquetar una aplicación junto con sus dependencias dentro de contenedores. Esto reduce las diferencias entre los entornos de desarrollo, pruebas y despliegue, debido a que cada servicio se ejecuta con una configuración definida.

Docker Compose permite describir varios servicios dentro de un archivo denominado `docker-compose.yml`. Para la arquitectura propuesta se consideraron cuatro servicios principales:

1. **Frontend:** aplicación Angular servida mediante Nginx.
2. **Backend:** API REST construida con Spring Boot.
3. **PostgreSQL:** almacenamiento persistente de los datos.
4. **Redis:** revocación de tokens y almacenamiento temporal.

Cada servicio puede declarar variables de entorno, puertos, volúmenes, redes y dependencias. Los *health checks* permiten comprobar si un contenedor está preparado para recibir solicitudes antes de iniciar los servicios que dependen de él [12].

Por ejemplo, el backend debe esperar a que PostgreSQL y Redis se encuentren saludables antes de intentar establecer las conexiones correspondientes. Esta configuración contribuye a que el sistema pueda iniciarse mediante un único comando:

```
docker compose up --build
```

En la presente documentación, Docker Compose se planteó como el mecanismo de despliegue integrado para ejecutar la aplicación y sus componentes de infraestructura de forma reproducible.

3. Diseño del sistema

3.1. Arquitectura general del sistema

El Sistema de Gestión de Pre-Sustentaciones de Trabajos de Titulación fue estructurado mediante una arquitectura cliente-servidor organizada en diferentes contenedores tecnológicos. Esta separación permite distribuir las responsabilidades de presentación, lógica de negocio, seguridad y persistencia de datos.

El usuario accede al sistema mediante una aplicación web desarrollada con Angular. El frontend presenta los formularios, paneles y opciones disponibles según el rol del usuario. Las solicitudes se envían al backend mediante el protocolo HTTP y utilizando datos en formato JSON.

El backend fue desarrollado con Java y Spring Boot. Este contenedor implementa los controladores REST, servicios, repositorios, reglas del negocio y mecanismos de seguridad. Spring Security intercepta las solicitudes protegidas y verifica la presencia y validez del token JWT antes de permitir el acceso a los recursos.

La persistencia de la información se realiza en PostgreSQL. La comunicación entre Spring Boot y PostgreSQL se gestiona mediante Spring Data JPA, Hibernate y el controlador JDBC correspondiente.

La arquitectura también contempla Redis como servicio de almacenamiento temporal para la revocación de tokens JWT y futuras funciones de caché. En la presente versión, Redis se documentó como parte del diseño de seguridad propuesto para la ampliación del módulo de autenticación.

El frontend puede ser servido mediante Nginx en el entorno de despliegue. Nginx entrega los archivos compilados de Angular y puede actuar como intermediario entre el navegador y la API REST.

3.2. Modelo C4 Nivel 2: contenedores

La Figura 1 presenta los principales contenedores de la aplicación y las relaciones existentes entre el usuario, el frontend Angular, el backend Spring Boot, PostgreSQL, Redis y Nginx.

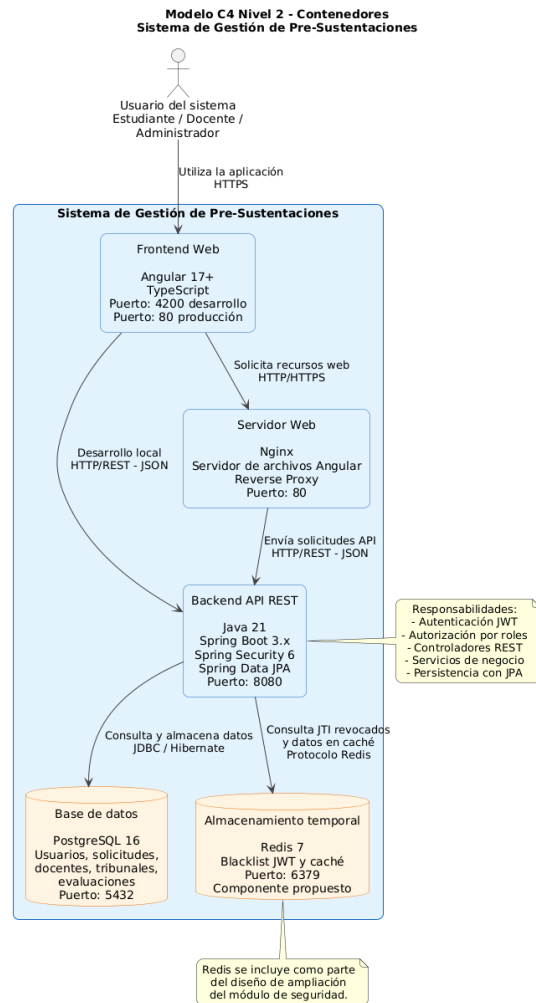


Figura 1: Modelo C4 Nivel 2 del Sistema de Gestión de Pre-Sustentaciones

3.3. Diagrama de clases UML refactorizado

El diagrama de clases representa la organización estructural del backend desarrollado con Spring Boot. Debido a la cantidad de clases implementadas, el modelo se dividió en dos partes para conservar la legibilidad. La primera parte presenta las entidades del dominio, sus atributos con tipos Java, métodos relevantes y relaciones de asociación. La segunda parte representa la interacción entre DTO, repositorios, servicios, controladores, componentes de seguridad y clases de configuración.

Las entidades se encuentran dentro del paquete `entities` y utilizan anotaciones de Jakarta Persistence para establecer el mapeo con PostgreSQL. Los atributos se declararon con visibilidad privada, mientras que los métodos de negocio relevantes se representaron con visibilidad pública o protegida según su implementación.

La entidad `Solicitud` constituye el elemento central del modelo. Esta se relaciona con el estudiante que inicia el proceso y con los componentes utilizados posteriormente para gestionar el anteproyecto, tutor, jurado, cronograma, evaluación y acta. El modelo también incorpora la gestión de rúbricas, criterios de evaluación, tutorías, mensajes y notificaciones.

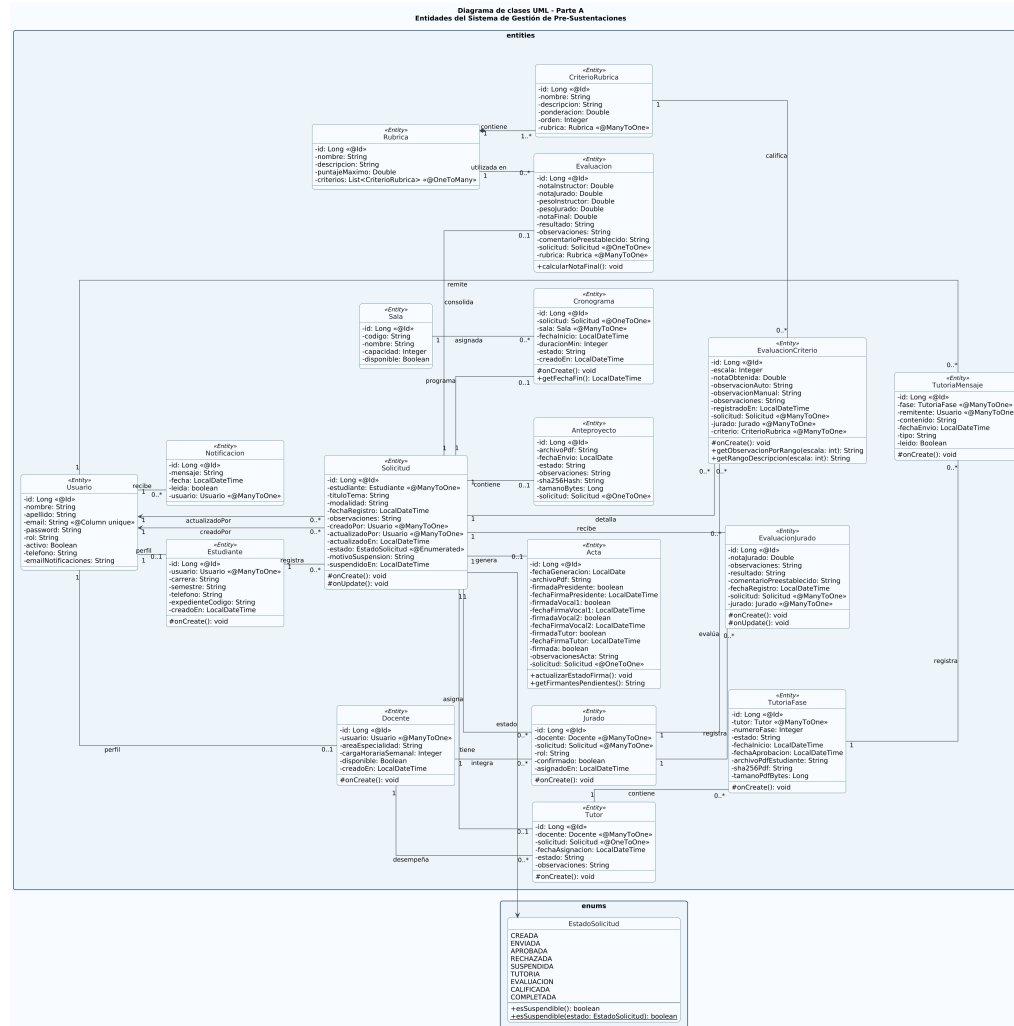


Figura 2: Diagrama de clases UML de las entidades del sistema

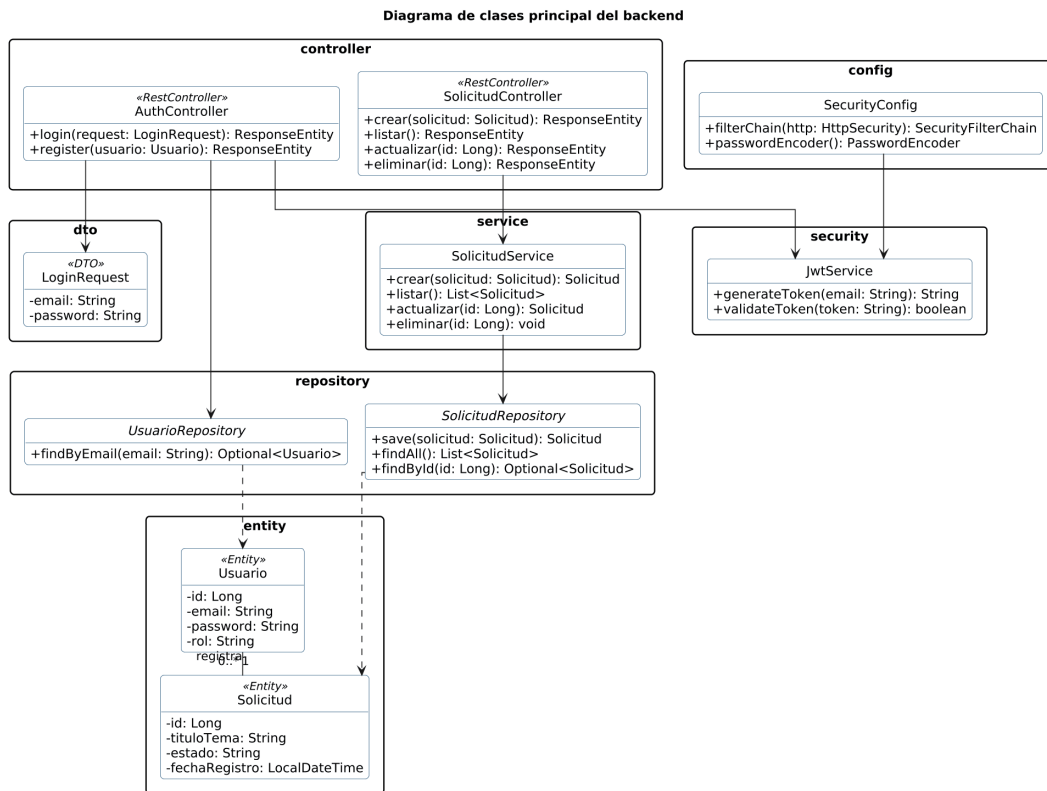


Figura 3: Diagrama de clases principal del backend

El diagrama de clases presentado en la Figura 3 muestra de forma simplificada las clases principales utilizadas en los módulos de autenticación JWT y gestión de solicitudes. El modelo se encuentra organizado por paquetes, con el propósito de separar las responsabilidades del backend y facilitar el mantenimiento del código.

El paquete **entity** contiene las entidades **Usuario** y **Solicitud**, las cuales representan la información persistida en PostgreSQL. La entidad **Usuario** almacena los datos necesarios para la autenticación y el control de roles, mientras que **Solicitud** representa el registro principal del proceso de pre-sustentación.

El paquete **repository** contiene las interfaces encargadas del acceso a datos. **UsuarioRepository** permite buscar usuarios mediante su correo electrónico y **SolicitudRepository** proporciona las operaciones necesarias para guardar, listar y consultar solicitudes. Estas interfaces utilizan Spring Data JPA, evitando la construcción manual y concatenación de sentencias SQL.

La gestión de solicitudes se realiza mediante **SolicitudController** y **SolicitudService**. El controlador recibe las solicitudes HTTP y delega las operaciones de creación, consulta, actualización y eliminación al servicio, donde se concentra la lógica del negocio.

El proceso de autenticación inicia en **AuthController**, que recibe las credenciales mediante **LoginRequest**, consulta al usuario y utiliza **JwtService** para generar o validar el token de acceso. Finalmente, **SecurityConfig** establece las reglas de seguridad, el codificador de contraseñas y la protección de los endpoints. Las flechas del diagrama representan las dependencias y asociaciones existentes entre las diferentes capas del backend.

3.4. Correspondencia entre Java y PostgreSQL

El mapeo objeto-relacional permite establecer una correspondencia entre los atributos declarados en las entidades Java y las columnas almacenadas en PostgreSQL. Hibernate interpreta las anotaciones JPA y transforma los tipos utilizados por el backend en tipos compatibles con la base de datos.

Las tablas siguientes presentan la correspondencia de las entidades principales utilizadas en los módulos de autenticación y gestión de solicitudes.

3.4.1. Entidad Usuario

Cuadro 1: Correspondencia Java–PostgreSQL de la entidad Usuario

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricción y descripción
id	Long	BIGSERIAL	Clave primaria autogenerada para identificar de manera única al usuario.
nombre	String	VARCHAR(100)	Nombre del usuario. Campo obligatorio.
apellido	String	VARCHAR(100)	Apellido del usuario. Campo obligatorio.
email	String	VARCHAR(255)	Correo utilizado para el inicio de sesión. Debe ser único y no nulo.
password	String	VARCHAR(255)	Almacena el hash de la contraseña generado mediante BCrypt. No debe devolverse en las respuestas JSON.
rol	String	VARCHAR(30)	Define los permisos del usuario dentro del sistema, por ejemplo, estudiante, docente o administrador.
activo	Boolean	BOOLEAN	Indica si el usuario puede acceder al sistema. Su valor predeterminado es verdadero.
telefono	String	VARCHAR(20)	Número de contacto del usuario. Puede ser opcional.
emailNotificaciones	String	VARCHAR(255)	Correo alternativo utilizado para el envío de notificaciones académicas.

3.4.2. Entidad Solicitud

Cuadro 2: Correspondencia Java-PostgreSQL de la entidad Solicitud

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricción y descripción
id	Long	BIGSERIAL	Clave primaria autogenerada de la solicitud.
estudiante	Estudiante	BIGINT	Clave foránea que relaciona la solicitud con el estudiante que inició el proceso.
tituloTema	String	VARCHAR(255)	Título propuesto para el trabajo de titulación. Campo obligatorio.
modalidad	String	VARCHAR(50)	Modalidad seleccionada para el trabajo de titulación.
fechaRegistro	LocalDateTime	TIMESTAMPTZ	Fecha y hora en que fue registrada la solicitud.
observaciones	String	TEXT	Comentarios generales relacionados con la solicitud.
estado	EstadoSolicitud	VARCHAR(30)	Estado actual del proceso, como creada, enviada, aprobada, rechazada o suspendida.
creadoPor	Usuario	BIGINT	Clave foránea del usuario que registró la solicitud.
actualizadoPor	Usuario	BIGINT	Clave foránea del último usuario que modificó la solicitud.
actualizadoEn	LocalDateTime	TIMESTAMPTZ	Fecha y hora de la última actualización del registro.
motivoSuspension	String	TEXT	Motivo por el cual la solicitud fue suspendida. Puede ser nulo.
suspendidoEn	LocalDateTime	TIMESTAMPTZ	Fecha y hora en que se realizó la suspensión. Puede ser nulo.

La correspondencia presentada permite observar cómo los atributos definidos en las entidades Java se transforman en columnas PostgreSQL. Las relaciones con otras entidades se almacenan mediante claves foráneas de tipo **BIGINT**, mientras que los campos de fecha utilizan **TIMESTAMPTZ** para conservar la información temporal. Las restricciones de unicidad, obligatoriedad y validación complementan las reglas aplicadas desde el backend.

3.5. Diagrama Entidad-Relación

El Diagrama Entidad-Relación presenta la estructura lógica de las tablas principales utilizadas en el Sistema de Gestión de Pre-Sustentaciones. A diferencia del diagrama de clases, este modelo se concentra en la perspectiva de persistencia, mostrando las claves primarias, claves foráneas, tipos de datos y cardinalidades existentes entre las tablas.

La tabla **solicitud** constituye el elemento central del modelo, debido a que registra el proceso iniciado por un estudiante. Cada solicitud puede relacionarse con un anteproyecto, un tutor, varios miembros del jurado, un cronograma, una evaluación y un acta. La información de autenticación se administra mediante la tabla **usuario**, la cual también se relaciona con los perfiles de estudiante y docente.

Las relaciones se establecen mediante claves foráneas de tipo **BIGINT**. Esto permite mantener la integridad referencial y evitar registros que no se encuentren asociados con una entidad válida. El modelo también incorpora la tabla **notificacion**, utilizada para comunicar cambios o eventos relevantes a los usuarios del sistema.

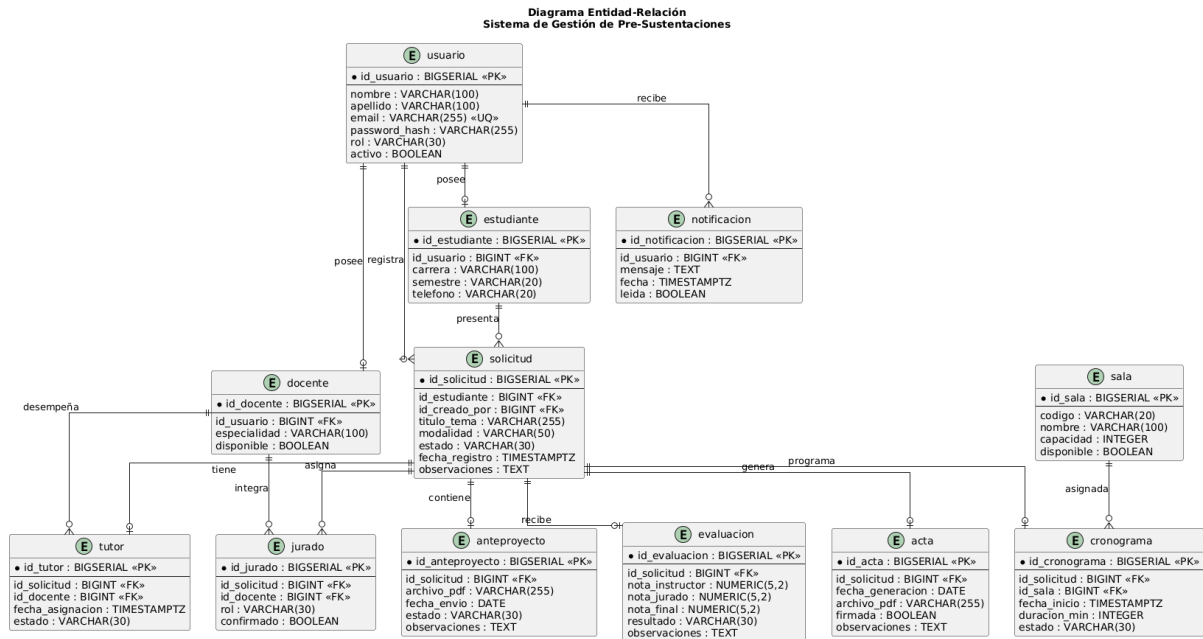


Figura 4: Diagrama Entidad-Relación del sistema

3.6. Diccionario de datos

El diccionario de datos describe la estructura de las tablas principales utilizadas en la base de datos. Para cada columna se especifica su tipo de dato en PostgreSQL, si admite valores nulos, su valor predeterminado y la función que cumple dentro del sistema.

3.6.1. Tabla usuario

La tabla **usuario** almacena la información necesaria para identificar, autenticar y autorizar a las personas que acceden al sistema.

Cuadro 3: Diccionario de datos de la tabla usuario

Campo	Tipo	Nulo	Predeterminado	Descripción
id_usuario	BIGSERIAL	No	Secuencia automática	Identificador único y clave primaria del usuario.
nombre	VARCHAR(100)	No	Sin valor	Nombre del usuario registrado en el sistema.
apellido	VARCHAR(100)	No	Sin valor	Apellido del usuario.
email	VARCHAR(255)	No	Sin valor	Correo utilizado para el inicio de sesión. Debe ser único.
password_hash	VARCHAR(255)	No	Sin valor	Hash de la contraseña generado mediante BCrypt.
rol	VARCHAR(30)	No	ROLE_USER	Rol utilizado para determinar los permisos del usuario.
activo	BOOLEAN	No	TRUE	Indica si el usuario se encuentra habilitado para acceder al sistema.
telefono	VARCHAR(20)	Sí	NULL	Número telefónico utilizado como información de contacto.
email_notificaciones	VARCHAR(255)	Sí	NULL	Correo alternativo utilizado para recibir notificaciones.
creado_en	TIMESTAMPZ	No	NOW()	Fecha y hora de creación del registro.

Campo	Tipo	Nulo	Predeterminad	Descripción
actualizado_en	TIMESTAMPTZ	No	NOW()	Fecha y hora de la última actualización.

3.6.2. Tabla solicitud

La tabla `solicitud` registra las solicitudes de pre-sustentación generadas por los estudiantes y conserva la información necesaria para realizar su seguimiento.

Cuadro 4: Diccionario de datos de la tabla solicitud

Campo	Tipo	Nulo	Predeterminad	Descripción
id_solicitud	BIGSERIAL	No	Secuencia automática	Identificador único y clave primaria de la solicitud.
id_estudiante	BIGINT	No	Sin valor	Clave foránea que identifica al estudiante propietario de la solicitud.
id_creado_por	BIGINT	No	Sin valor	Clave foránea del usuario que realizó el registro.
id_actualizado_por	BIGINT	Sí	NULL	Clave foránea del último usuario que modificó la solicitud.
titulo_tema	VARCHAR(255)	No	Sin valor	Título propuesto para el trabajo de titulación.
modalidad	VARCHAR(50)	No	Sin valor	Modalidad seleccionada para el desarrollo del trabajo.
estado	VARCHAR(30)	No	CREADA	Estado actual de la solicitud dentro del proceso académico.
fecha_registro	TIMESTAMPTZ	No	NOW()	Fecha y hora en que se registró la solicitud.
observaciones	TEXT	Sí	NULL	Comentarios o indicaciones relacionadas con la solicitud.
motivo_suspension	TEXT	Sí	NULL	Descripción del motivo por el cual fue suspendido el proceso.
suspendido_en	TIMESTAMPTZ	Sí	NULL	Fecha y hora en que fue aplicada la suspensión.
actualizado_en	TIMESTAMPTZ	No	NOW()	Fecha y hora correspondiente a la última modificación.

El diccionario de datos permite identificar las restricciones aplicadas sobre la información y facilita la comprensión del esquema utilizado por el backend. Los campos definidos como no nulos representan datos obligatorios, mientras que las claves foráneas mantienen las relaciones entre usuarios, estudiantes y solicitudes. Los campos temporales permiten conservar la trazabilidad de los registros y sus modificaciones.

3.7. Diagrama de secuencia de autenticación JWT

El diagrama de secuencia representa la interacción entre el usuario, el frontend Angular, los componentes de seguridad del backend y la base de datos PostgreSQL durante el proceso de autenticación.

El flujo comienza cuando el usuario ingresa su correo electrónico y contraseña en la interfaz Angular. El frontend envía las credenciales al endpoint `/api/auth/login`, donde el controlador delega la autenticación al componente `AuthenticationManager`.

Spring Security utiliza `UserDetailsService` para localizar al usuario mediante `UsuarioRepository`. El repositorio consulta PostgreSQL y devuelve la información correspondiente. Posteriormente, `PasswordEncoder` compara la contraseña recibida con el hash almacenado en la base de datos.

Cuando las credenciales son válidas, `JwtService` genera un token firmado que es devuelto al frontend. Angular utiliza este token en las solicitudes posteriores, agregándolo al encabezado `Authorization` con el esquema `Bearer`.

En cada petición protegida, **JwtAuthFilter** extrae y valida el token antes de establecer la autenticación en **SecurityContextHolder**. El diseño también contempla la consulta de Redis para comprobar que el identificador del token no se encuentre dentro de una lista de revocación.

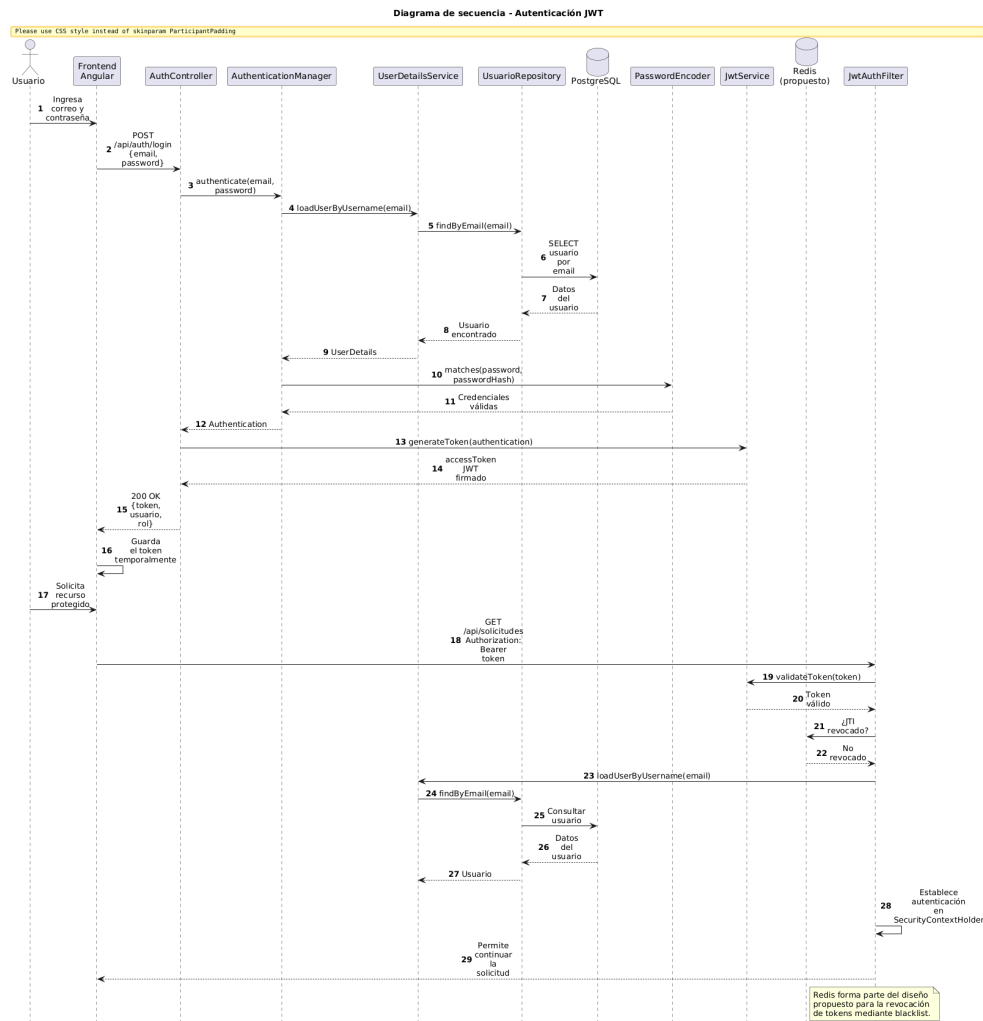


Figura 5: Diagrama de secuencia del proceso de autenticación JWT

3.8. Migración inicial de base de datos con Flyway

Para establecer una estructura reproducible de la base de datos se diseñó una migración inicial denominada **V1__schema_inicial.sql**. Este archivo contiene la definición de las tablas principales relacionadas con la autenticación y la gestión de solicitudes.

La migración utiliza tipos propios de PostgreSQL, restricciones de integridad, índices y un trigger encargado de actualizar automáticamente el campo **actualizado_en**. Debido a que la integración completa de Flyway se encuentra planteada como una ampliación técnica, el siguiente script representa la estructura propuesta para la inicialización del esquema.

Para controlar la creación del esquema se propuso la migración **V1__schema_inicial.sql**. El siguiente fragmento presenta la estructura principal de la tabla de usuarios, sus restricciones y el trigger encargado de actualizar automáticamente la fecha de modificación.

```

1 CREATE TABLE usuarios (
2   id BIGSERIAL PRIMARY KEY,
3   nombre VARCHAR(100) NOT NULL,
4   email VARCHAR(255) NOT NULL,
5   password_hash VARCHAR(255) NOT NULL,
6   rol VARCHAR(30) NOT NULL DEFAULT 'ROLE_USER',
7   activo BOOLEAN NOT NULL DEFAULT TRUE,

```

```

8      creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
9      actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
10
11     CONSTRAINT chk_usuario_rol
12     CHECK (rol IN (
13         'ROLE_USER',
14         'ROLE_ESTUDIANTE',
15         'ROLE_DOCENTE',
16         'ROLE_ADMIN'
17     ))
18 );
19
20     CREATE UNIQUE INDEX idx_usuarios_email
21     ON usuarios(email);
22
23     CREATE OR REPLACE FUNCTION set_actualizado_en()
24     RETURNS TRIGGER AS $$
25     BEGIN
26         NEW.actualizado_en = NOW();
27         RETURN NEW;
28     END;
29     $$ LANGUAGE plpgsql;
30
31     CREATE TRIGGER trg_usuarios_actualizado_en
32     BEFORE UPDATE ON usuarios
33     FOR EACH ROW
34     EXECUTE FUNCTION set_actualizado_en();

```

Listing 1: Fragmento de la migración inicial con Flyway

El identificador se genera automáticamente mediante `BIGSERIAL`. Los campos temporales utilizan `TIMESTAMPTZ` y el estado del usuario se representa mediante `BOOLEAN`. La restricción `CHECK` controla los roles permitidos, mientras que el índice único evita el registro de correos duplicados. Finalmente, el trigger actualiza el campo `actualizado_en` cuando se modifica un usuario.

El uso de `BIGSERIAL` permite generar automáticamente los identificadores numéricos de las tablas. Los campos temporales se definieron mediante `TIMESTAMPTZ`, mientras que el atributo `activo` utiliza el tipo `BOOLEAN` para representar la habilitación lógica de los registros.

La restricción `CHECK` limita los valores permitidos para los roles y estados de las solicitudes. El índice único creado sobre el correo electrónico evita el registro de usuarios duplicados. Finalmente, la función `set_actualizado_en()` y sus triggers actualizan la fecha de modificación cada vez que cambia un registro.

4. Implementación

4.1. Estructura del repositorio

El proyecto se organizó mediante una estructura que separa el frontend Angular, el backend Spring Boot, los recursos públicos, los archivos de configuración y los documentos almacenados por el sistema. Esta distribución facilita la localización de los componentes y mantiene separadas las responsabilidades de presentación, lógica de negocio y persistencia.

La estructura resumida del repositorio es la siguiente:

```

ProyectoWebPresustentaciones/
|
|-- backend/
|   |-- backend-mod/
|       |-- src/main/java/
|           |-- ec/edu/uteq/presustentaciones/
|               |-- config/
|               |-- controllers/
|               |-- dto/
|               |-- entities/

```

```

|         |         |-- enums/
|         |         |-- repositories/
|         |         |-- security/
|         |         '-- services/
|         |
|         |-- src/main/resources/
|         |-- pom.xml
|         |-- .env.example
|         '-- README.md
|
|-- src/
|   |-- app/
|     |-- components/
|     |-- guards/
|     |-- interceptors/
|     |-- models/
|     '-- services/
|     |
|     |-- main.ts
|     '-- styles.css
|
|-- public/
|   '-- img/
|
|-- uploads/
|   |-- actas/
|   '-- anteproyectos/
|
|-- angular.json
|-- package.json
|-- tsconfig.json
'-- README.md

```

Cuadro 5: Descripción de las carpetas principales del repositorio

Ruta	Descripción
backend/backend-mod/	Contiene el proyecto principal del backend desarrollado con Spring Boot y Maven.
src/main/java/	Almacena el código Java organizado en paquetes según las capas de la aplicación.
controllers/	Contiene los controladores REST que reciben las solicitudes HTTP del frontend.
services/	Incluye la lógica del negocio y coordina las operaciones realizadas por los controladores.
repositories/	Contiene las interfaces de acceso a datos basadas en Spring Data JPA.
entities/	Define las entidades Java que se relacionan con las tablas de PostgreSQL.
security/	Contiene los componentes encargados de generar, validar y procesar los tokens JWT.
config/	Incluye la configuración de Spring Security, CORS y otros componentes del backend.
src/app/	Contiene la aplicación frontend desarrollada con Angular.
components/	Agrupar las vistas y componentes correspondientes a estudiantes, docentes y administradores.
guards/ y interceptors/	Protegen las rutas y agregan el token JWT a las solicitudes HTTP.
services/	Contiene los servicios Angular que se comunican con los endpoints del backend.
public/img/	Almacena logotipos, íconos e imágenes utilizadas en la interfaz.
uploads/	Conserva los archivos de anteproyectos y actas generados o cargados en el sistema.

Esta estructura evidencia la separación entre el frontend y el backend. En Spring Boot, el código se encuentra distribuido por capas, mientras que Angular organiza sus funciones mediante componentes, servicios, modelos, interceptores y guardias de navegación.

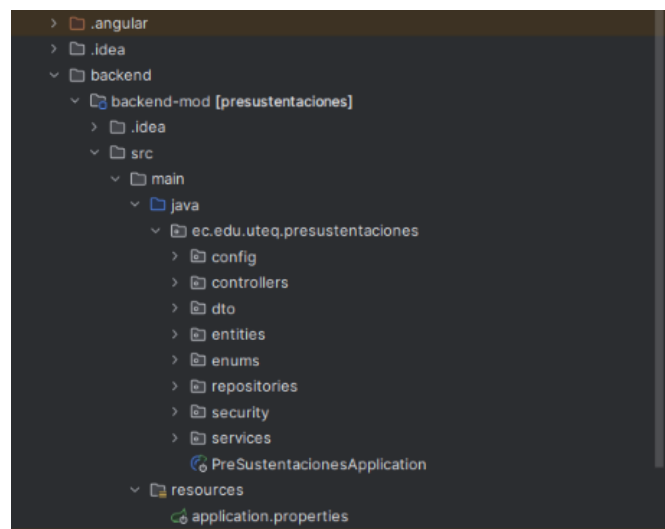


Figura 6: Estructura del backend Spring Boot en el proyecto

La Figura 6 muestra la organización interna del backend Spring Boot dentro del paquete principal

del proyecto. En ella se observan los paquetes utilizados para configuración, controladores, objetos de transferencia de datos, entidades, enumeraciones, repositorios, seguridad y servicios, además del archivo de configuración `application.properties`.

4.2. Tecnologías utilizadas

El sistema fue desarrollado mediante tecnologías orientadas a la construcción de aplicaciones web modernas. El frontend se implementó con Angular, mientras que el backend se construyó con Java y Spring Boot. PostgreSQL se utilizó para almacenar la información y Spring Data JPA junto con Hibernate permitieron realizar el mapeo entre las entidades Java y las tablas de la base de datos.

Cuadro 6: Tecnologías utilizadas en el proyecto

Tecnología	Función dentro del proyecto
Java	Lenguaje utilizado para implementar el backend y la lógica del negocio.
Spring Boot	Framework utilizado para desarrollar la API REST y configurar los componentes del backend.
Spring Security	Gestiona la autenticación, autorización, protección de rutas y validación de usuarios.
JSON Web Token	Permite generar tokens de acceso para autenticar las solicitudes protegidas.
Spring Data JPA	Proporciona repositorios para consultar y almacenar información sin construir SQL manual.
Hibernate	Realiza el mapeo objeto-relacional entre las entidades Java y PostgreSQL.
PostgreSQL	Sistema gestor de base de datos utilizado para almacenar usuarios, solicitudes y demás registros.
Angular	Framework utilizado para implementar la interfaz web del sistema.
Swagger/OpenAPI	Permite visualizar y documentar los endpoints disponibles en el backend.
Maven	Herramienta utilizada para administrar dependencias y compilar el backend.

4.3. Configuración del backend

La configuración principal del backend se encuentra en el archivo `application.properties`, ubicado en la carpeta `src/main/resources`. En este archivo se definen la conexión con PostgreSQL, el comportamiento de Hibernate y los parámetros utilizados para generar los tokens JWT.

Para evitar exponer información sensible, las credenciales y la clave secreta deben administrarse mediante variables de entorno. El siguiente fragmento presenta una configuración resumida:

Listing 2: Configuración principal de Spring Boot

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}

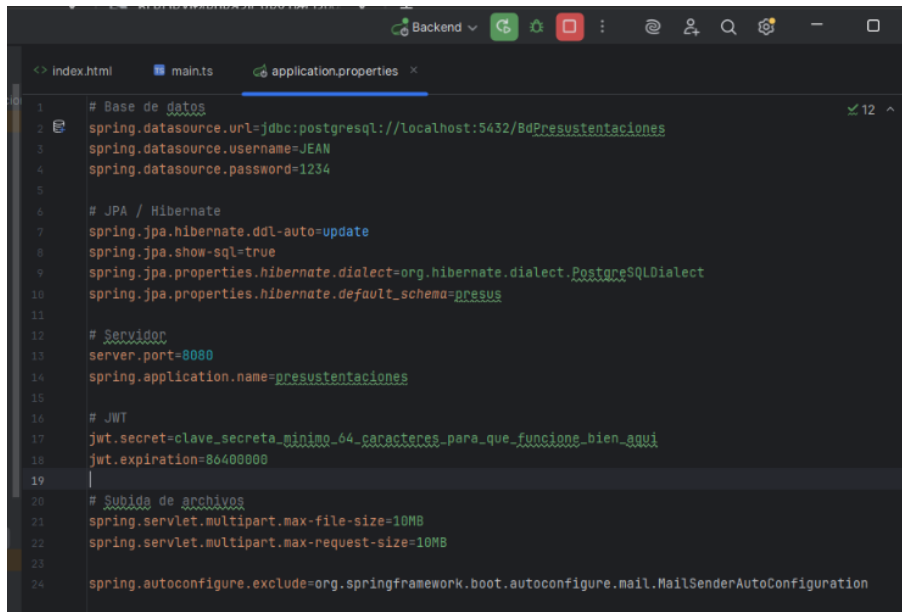
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false

jwt.secret=${JWT_SECRET}
jwt.expiration=86400000

server.port=8080
```

La propiedad `spring.datasource.url` establece la dirección de PostgreSQL, mientras que las propiedades de usuario y contraseña permiten realizar la autenticación con la base de datos. La configuración de JPA determina el comportamiento de Hibernate y las propiedades JWT definen la clave de firma y el tiempo de expiración del token.

Aunque la guía recomienda una duración de una hora para el token, la configuración actual del proyecto utiliza un tiempo de expiración de veinticuatro horas. Esta diferencia se identifica como una mejora de seguridad para futuras versiones.



```
1 # Base de datos
2 spring.datasource.url=jdbc:postgresql://localhost:5432/8dPresustentaciones
3 spring.datasource.username=JEAN
4 spring.datasource.password=1234
5
6 # JPA / Hibernate
7 spring.jpa.hibernate.ddl-auto=update
8 spring.jpa.show-sql=true
9 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
10 spring.jpa.properties.hibernate.default_schema=presus
11
12 # Servidor
13 server.port=8080
14 spring.application.name=presustentaciones
15
16 # JWT
17 jwt.secret=clave_secreta_mining_64_caracteres_para_que_funcione_bien_aqui
18 jwt.expiration=86400000
19
20 # Subida de archivos
21 spring.servlet.multipart.max-file-size=10MB
22 spring.servlet.multipart.max-request-size=10MB
23
24 spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.mail.MailSenderAutoConfiguration
```

Figura 7: Configuración principal del backend Spring Boot

4.4. Implementación del inicio de sesión

El módulo de autenticación permite verificar las credenciales de los usuarios antes de conceder acceso a los recursos protegidos. El proceso inicia cuando el frontend Angular envía una solicitud HTTP al endpoint de inicio de sesión con el correo electrónico y la contraseña.

El controlador `AuthController` recibe los datos mediante el objeto `LoginRequest` y delega la autenticación al componente `AuthenticationManager`. Spring Security utiliza `UserDetailsService` para buscar al usuario por su correo electrónico y comparar la contraseña ingresada con el hash almacenado en PostgreSQL.

Cuando las credenciales son correctas, el componente `JwtTokenProvider` genera un token JWT firmado. Este token es devuelto al frontend junto con información básica del usuario, como su identificador, nombre, correo y rol.

El flujo simplificado del inicio de sesión se presenta a continuación:

1. Angular envía el correo y la contraseña.
2. `AuthController` recibe la petición.
3. `AuthenticationManager` valida las credenciales.
4. `UsuarioRepository` consulta al usuario en PostgreSQL.
5. Spring Security compara la contraseña ingresada con el hash almacenado.
6. `JwtTokenProvider` genera el token de acceso.
7. El backend devuelve el JWT al frontend.

El endpoint principal utilizado para esta operación es:

POST /api/auth/login

Un ejemplo resumido de la petición es:

Listing 3: Datos enviados al endpoint de inicio de sesion

```
{
    "email": "usuario@uteq.edu.ec",
    "password": "*****"
}
```

La respuesta contiene el token que debe utilizarse en las solicitudes posteriores:

Listing 4: Respuesta resumida del inicio de sesion

```
{
    "token": "eyJhbGciOiJIUzI1NiJ9...",
    "id": 1,
    "email": "usuario@uteq.edu.ec",
    "rol": "ROLE_USER"
}
```

4.5. Generación del token JWT

La clase `JwtTokenProvider` concentra las operaciones relacionadas con la generación y validación de los tokens. Entre sus principales responsabilidades se encuentran obtener los datos del usuario autenticado, establecer la fecha de emisión, definir la expiración y firmar el contenido mediante una clave secreta.

Los métodos principales representados en el proyecto son:

```
generateToken(authentication)
getUsernameFromToken(token)
validateToken(token)
```

El token generado contiene la información mínima necesaria para identificar al usuario y comprobar su periodo de validez. Debido a que el contenido del JWT puede ser decodificado, no se almacenan contraseñas ni datos sensibles dentro de su payload.

4.6. Implementación del filtro JWT

La clase `JwtAuthenticationFilter` intercepta cada solicitud dirigida a los endpoints protegidos. Este componente extiende `OncePerRequestFilter`, lo que garantiza que el filtro se ejecute una sola vez por petición.

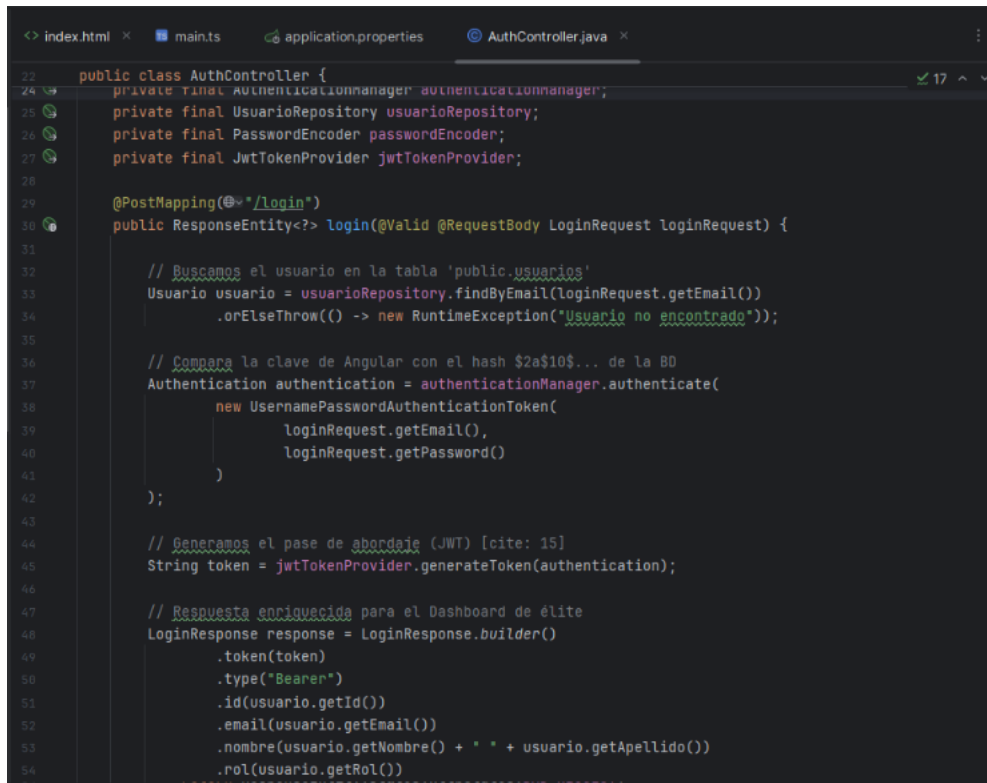
El proceso realizado por el filtro es el siguiente:

1. Lee el encabezado HTTP `Authorization`.
2. Verifica que el valor comience con la palabra `Bearer`.
3. Extrae el token JWT.
4. Valida la firma y la fecha de expiración.
5. Obtiene el correo o nombre de usuario almacenado en el token.
6. Consulta los datos del usuario mediante `UserDetailsService`.
7. Establece la autenticación en `SecurityContextHolder`.
8. Permite continuar la solicitud hacia el controlador correspondiente.

El encabezado utilizado tiene la siguiente estructura:

```
Authorization: Bearer <token>
```

La configuración de seguridad incorpora el filtro JWT antes del filtro estándar de autenticación por usuario y contraseña. De esta forma, las peticiones que contienen un token válido pueden acceder a los recursos protegidos sin crear una sesión tradicional en el servidor.

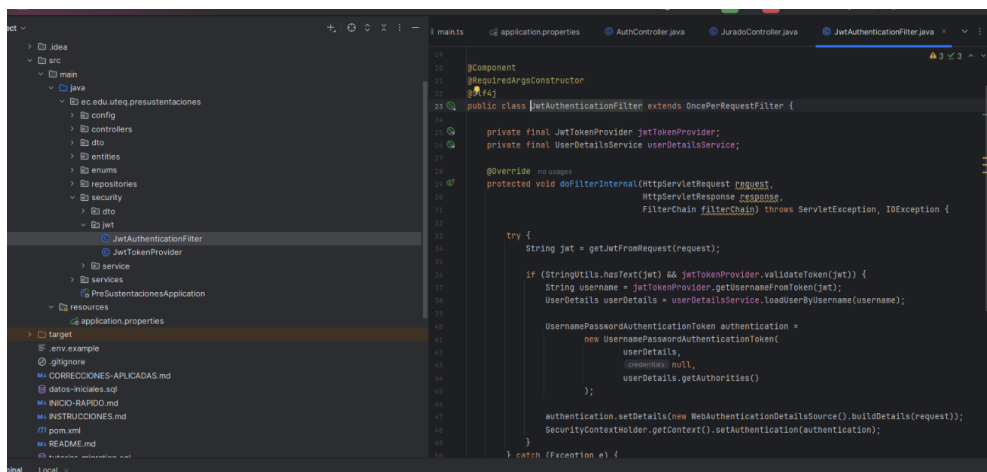


```

22 public class AuthController {
23     private final AuthenticationManager authenticationManager;
24     private final UsuarioRepository usuarioRepository;
25     private final PasswordEncoder passwordEncoder;
26     private final JwtTokenProvider jwtTokenProvider;
27
28
29     @PostMapping("/login")
30     public ResponseEntity<?> login(@Valid @RequestBody LoginRequest loginRequest) {
31
32         // Buscamos el usuario en la tabla 'public.usuarios'
33         Usuario usuario = usuarioRepository.findByEmail(loginRequest.getEmail())
34             .orElseThrow(() -> new RuntimeException("Usuario no encontrado"));
35
36         // Comparamos la clave de Angular con el hash $2a$10$... de la BD
37         Authentication authentication = authenticationManager.authenticate(
38             new UsernamePasswordAuthenticationToken(
39                 loginRequest.getEmail(),
40                 loginRequest.getPassword()
41             )
42         );
43
44         // Generamos el pase de abordaje (JWT) [cite: 15]
45         String token = jwtTokenProvider.generateToken(authentication);
46
47         // Respuesta enriquecida para el Dashboard de elite
48         LoginResponse response = LoginResponse.builder()
49             .token(token)
50             .type("Bearer")
51             .id(usuario.getId())
52             .email(usuario.getEmail())
53             .nombre(usuario.getNombre() + " " + usuario.getApellido())
54             .rol(usuario.getRol())

```

Figura 8: Implementación del inicio de sesión en AuthController



```

18
19 @Component
20 @RequiredArgsConstructor
21 @Filter
22 public class JwtAuthenticationFilter extends OncePerRequestFilter {
23
24     private final JwtTokenProvider jwtTokenProvider;
25     private final UserDetailsService userDetailsService;
26
27
28     @Override
29     protected void doFilterInternal(HttpServletRequest request,
30                                   HttpServletResponse response,
31                                   FilterChain filterChain) throws ServletException, IOException {
32
33         try {
34             String jwt = getJwtFromRequest(request);
35
36             if (StringUtils.hasText(jwt) && jwtTokenProvider.validateToken(jwt)) {
37                 String username = jwtTokenProvider.getUsernameFromToken(jwt);
38                 UserDetails userDetails = userDetailsService.loadUserByUsername(username);
39
40                 UsernamePasswordAuthenticationToken authentication =
41                     new UsernamePasswordAuthenticationToken(
42                         userDetails,
43                         null,
44                         userDetails.getAuthorities()
45                     );
46
47                 authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
48                 SecurityContextHolder.getContext().setAuthentication(authentication);
49             }
50         } catch (Exception e) {

```

Figura 9: Implementación del filtro de autenticación JWT

4.7. Implementación del acceso a datos

El acceso a los datos del sistema se implementó mediante Spring Data JPA e Hibernate. Los controladores reciben las solicitudes HTTP, delegan las operaciones a los servicios y estos utilizan los repositorios para consultar o modificar la información almacenada en PostgreSQL.

La entidad **Solicitud** se consideró como una de las entidades principales del sistema, debido a que representa el inicio del proceso de pre-sustentación. Mediante su controlador se pueden registrar solicitudes, consultar los registros existentes y cambiar su estado durante el proceso académico.

La estructura utilizada para gestionar una solicitud sigue el siguiente flujo:

```
SolicitudController
|
v
SolicitudService
|
v
SolicitudRepository
|
v
PostgreSQL
```

Spring Data JPA proporciona métodos como `save()`, `findAll()` y `findById()`, evitando la concatenación manual de instrucciones SQL dentro del código Java.

4.8. Endpoints principales implementados

La Tabla 7 presenta los principales endpoints disponibles en el backend.

Cuadro 7: Endpoints principales del sistema

Método	Endpoint	Auth	Descripción
POST	/api/auth/login	No	Valida las credenciales y devuelve un token JWT.
POST	/api/auth/register	No	Registra un nuevo usuario en el sistema.
GET	/api/usuarios	JWT	Devuelve la lista de usuarios registrados.
GET	/api/usuarios/{id}	JWT	Consulta un usuario mediante su identificador.
POST	/api/usuarios	JWT	Crea un nuevo usuario desde el módulo administrativo.
PUT	/api/usuarios/{id}	JWT	Actualiza los datos de un usuario existente.
DELETE	/api/usuarios/{id}	JWT	Elimina o desactiva un usuario del sistema.
GET	/api/solicitudes	JWT	Lista las solicitudes registradas.
POST	/api/solicitudes	JWT	Registra una nueva solicitud de pre-sustentación.
GET	/api/solicitudes/{id}	JWT	Consulta la información de una solicitud específica.
PUT	/api/solicitudes/{id}	JWT	Actualiza la información de una solicitud.
PUT	/api/solicitudes/{id}/enviar	JWT	Cambia el estado de una solicitud a enviada.
PUT	/api/solicitudes/{id}/aprobar	JWT	Aprueba una solicitud registrada.
PUT	/api/solicitudes/{id}/rechazar	JWT	Rechaza una solicitud y registra las observaciones.

Los endpoints de autenticación son públicos, debido a que deben permitir el acceso inicial y el registro de usuarios. Los demás recursos se encuentran protegidos mediante Spring Security y requieren que el cliente envíe un token JWT válido en el encabezado **Authorization**.

4.9. Pruebas de integración con Postman

Para comprobar el funcionamiento de la autenticación y el acceso a los recursos protegidos, se realizaron pruebas manuales mediante Postman. Las evidencias muestran el inicio de sesión exitoso y la consulta de solicitudes utilizando el token JWT recibido.

4.9.1. Prueba de inicio de sesión

La Figura 10 muestra la petición enviada al endpoint de autenticación. El backend validó las credenciales del usuario y devolvió un token JWT junto con su identificador, correo electrónico, nombre y rol.

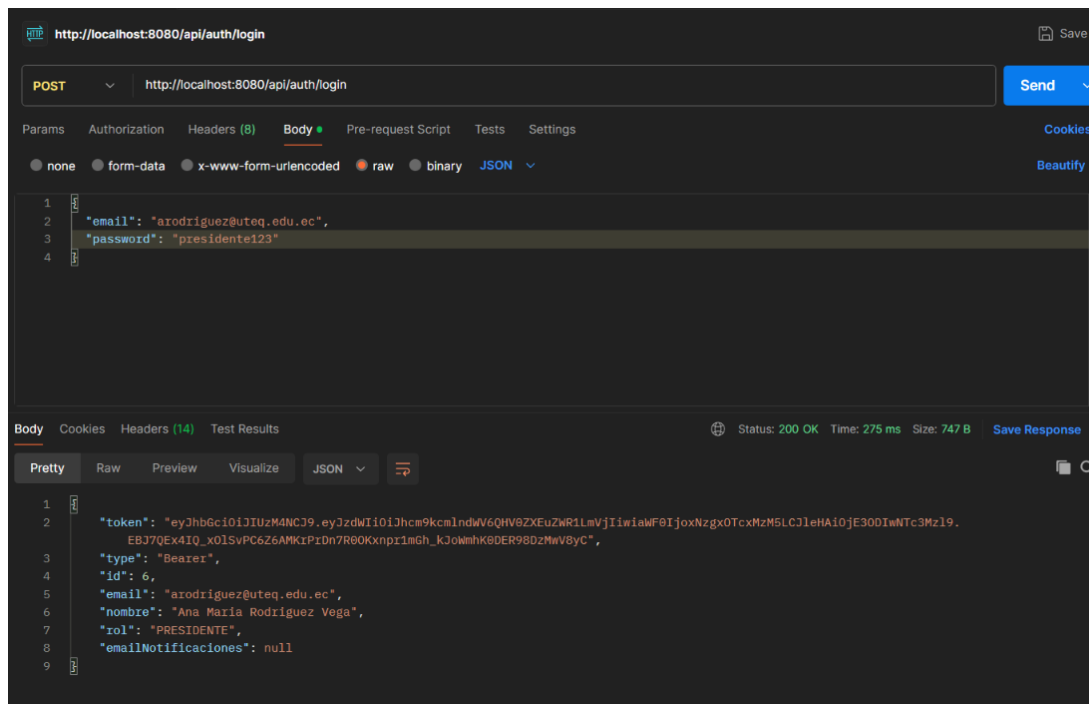


Figura 10: Inicio de sesión exitoso y generación del token JWT en Postman

4.9.2. Consulta de solicitudes con JWT

Después de obtener el token, este se agregó en la pestaña **Authorization** de Postman mediante el tipo **Bearer Token**. La Figura 11 presenta la consulta realizada sobre el endpoint protegido de solicitudes.

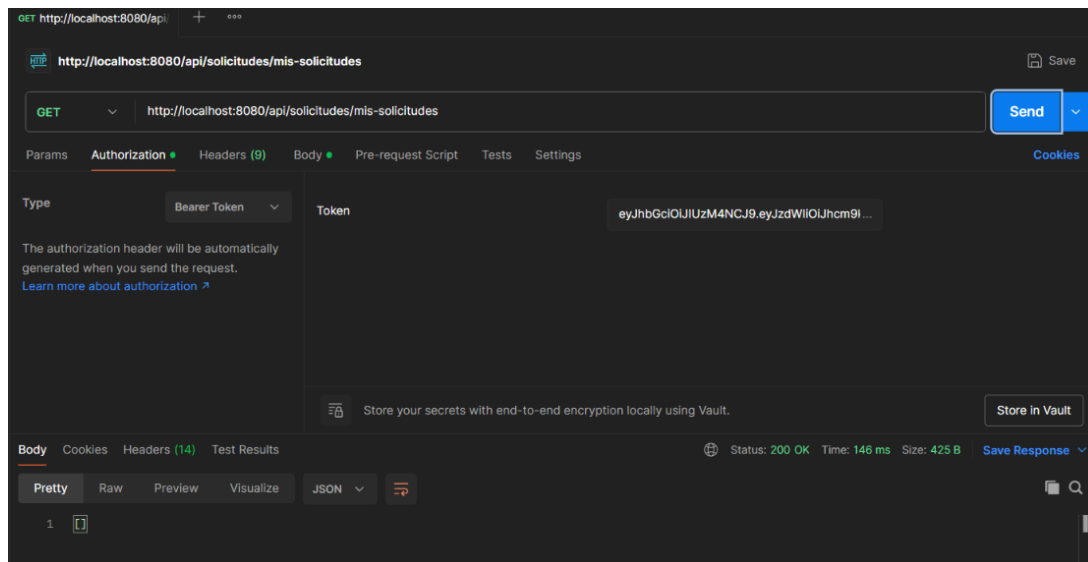


Figura 11: Consulta de solicitudes mediante autenticación Bearer JWT

Las pruebas permitieron comprobar que el endpoint de inicio de sesión genera un token JWT y que dicho token puede utilizarse para enviar solicitudes autenticadas al backend. La respuesta obtenida demuestra la comunicación entre Postman, Spring Security, los controladores REST y PostgreSQL.

4.10. Implementación del frontend Angular

El frontend del sistema fue desarrollado con Angular y se encarga de presentar las interfaces utilizadas por los diferentes usuarios. La aplicación consume los endpoints REST proporcionados por Spring Boot mediante solicitudes HTTP.

La autenticación en el frontend se organiza principalmente mediante el servicio **AuthService**, el interceptor JWT y el guard de rutas. Estos componentes permiten iniciar sesión, conservar temporalmente la información del usuario, adjuntar el token a las solicitudes y restringir el acceso a las vistas protegidas.

4.10.1. Servicio de autenticación

El servicio **AuthService** envía las credenciales al endpoint `/api/auth/login`. Cuando el backend devuelve una respuesta correcta, el servicio conserva el token JWT y los datos principales del usuario autenticado.

Las principales responsabilidades del servicio son:

- Enviar el correo y la contraseña al backend.
- Recibir el token JWT generado por Spring Security.
- Consultar si existe una sesión autenticada.
- Obtener el rol y los datos del usuario.
- Eliminar la información almacenada durante el cierre de sesión.

En la versión actual del proyecto, el token se conserva mediante **localStorage**. Aunque esta implementación permite mantener la sesión al recargar la página, se reconoce como mejora futura almacenar el token de acceso únicamente en memoria y utilizar una cookie **HttpOnly** para el refresh token.

4.10.2. Interceptor JWT

El interceptor HTTP se ejecuta antes de enviar una solicitud desde Angular hacia el backend. Su función principal consiste en obtener el token almacenado y clonar la petición para agregar el encabezado de autorización.

La estructura del encabezado enviado es:

Authorization: Bearer <token>

Gracias al interceptor, los componentes no necesitan agregar manualmente el token en cada solicitud. El backend recibe el encabezado y el filtro JWT valida su firma, expiración y usuario asociado.

4.10.3. Protección de rutas con AuthGuard

El guard de autenticación controla el acceso a las rutas protegidas de Angular. Antes de cargar una vista, verifica si existe un usuario autenticado y si dispone de los permisos necesarios.

Cuando no existe un token válido, el guard redirige al usuario hacia la pantalla de inicio de sesión. Esta validación evita que una persona acceda directamente mediante la URL a los paneles administrativos, estudiantiles o docentes.

4.10.4. Interfaz de inicio de sesión

La Figura 12 presenta la pantalla utilizada para ingresar las credenciales institucionales. La interfaz permite capturar el correo electrónico y la contraseña antes de enviar la solicitud al backend.



Figura 12: Interfaz de inicio de sesión desarrollada con Angular

La integración entre Angular y Spring Boot permite que el usuario inicie sesión desde la interfaz web y posteriormente acceda a los módulos correspondientes según su rol.

5. Pruebas

5.1. Casos de prueba de autenticación

Para comprobar el comportamiento del módulo de autenticación y de los endpoints protegidos, se definieron cinco casos de prueba principales. Las validaciones disponibles se realizaron mediante Postman, enviando solicitudes directamente al backend Spring Boot.

Cuadro 8: Casos de prueba del módulo de autenticación

N.º	Caso	Entrada	Resultado esperado	Resultado obtenido
1	Inicio de sesión correcto	Correo y contraseña válidos	Código 200 y generación de un token JWT	Validado mediante Postman. El backend devolvió el token y los datos del usuario.
2	Inicio de sesión con contraseña incorrecta	Correo válido y contraseña incorrecta	Código 401 o rechazo de autenticación	Caso definido para comprobar el rechazo de credenciales inválidas.
3	Registro con correo duplicado	Datos de registro con un correo existente	Mensaje de conflicto o validación de correo duplicado	Caso definido para comprobar la restricción de unicidad del correo.
4	Acceso protegido sin token	Petición a un endpoint protegido sin encabezado Authorization	Código 401 o 403 y acceso denegado	Validado mediante Postman. Spring Security rechazó la petición sin credenciales.
5	Acceso protegido con token válido	Petición con encabezado Bearer y JWT válido	Código 200 y acceso al recurso solicitado	Validado mediante la consulta de solicitudes desde Postman.

5.2. Resultados de las pruebas manuales

Las pruebas realizadas permitieron comprobar que el backend se encontraba disponible en el puerto 8080 y que Spring Security protegía los recursos que requerían autenticación. Una petición enviada sin token produjo una respuesta de acceso denegado, mientras que el inicio de sesión con credenciales válidas generó correctamente un JWT.

Posteriormente, el token recibido se incorporó como **Bearer Token** en la pestaña de autorización de Postman. Esta operación permitió realizar una solicitud autenticada hacia el módulo de solicitudes, confirmando que el filtro JWT procesó el encabezado y estableció el contexto de seguridad del usuario.

Durante la prueba de creación de solicitudes se identificó que el usuario autenticado poseía el rol **PRESIDENTE** y no tenía un perfil de estudiante asociado. Por esta razón, el backend devolvió el mensaje de validación correspondiente. Este resultado demuestra que el sistema controla la relación entre el usuario autenticado y el perfil requerido para registrar una solicitud.

5.3. Pruebas automatizadas propuestas

Para ampliar la cobertura del sistema se plantearon las siguientes pruebas automatizadas con JUnit 5 y MockMvc:

- `loginExitoso()`: verifica la autenticación con credenciales correctas.
- `loginClaveIncorrecta()`: comprueba el rechazo de una contraseña inválida.
- `registroEmailDuplicado()`: valida la restricción de correo único.
- `accesoSinToken()`: confirma la denegación de un endpoint protegido.

- `accesoConTokenValido()`: comprueba el acceso utilizando un JWT válido.

La automatización completa de estos casos y la generación de la evidencia de ejecución mediante `mvn test` se consideran actividades de fortalecimiento para la siguiente versión del proyecto.

5.4. Métricas de rendimiento

Las métricas de rendimiento permiten evaluar el tiempo requerido por el backend para procesar las solicitudes HTTP. Para esta entrega se utilizaron los tiempos mostrados por Postman después de ejecutar los endpoints principales.

El tiempo promedio representa la duración media de las solicitudes realizadas, mientras que el percentil 95, denominado P95, indica el tiempo máximo dentro del cual se completó aproximadamente el 95 % de las peticiones.

Cuadro 9: Métricas de rendimiento de los endpoints principales

Endpoint	Promedio	P95	Observación
POST /api/auth/login	Completar ms	Completar ms	Incluye la consulta del usuario, validación de credenciales y generación del JWT.
GET /api/solicitudes	Completar ms	Completar ms	Consulta protegida mediante token JWT y acceso a PostgreSQL.
POST /api/solicitudes	No medido	No medido	La prueba no fue completada debido a que el usuario autenticado no tenía un perfil de estudiante asociado.
GET /api/solicitudes con Redis	No disponible	No disponible	Redis se encuentra definido como ampliación técnica y no está activo en la versión actual.

Para calcular el tiempo promedio se utiliza la siguiente expresión:

$$T_{\text{promedio}} = \frac{T_1 + T_2 + T_3 + \dots + T_n}{n}$$

El percentil 95 se obtiene ordenando los tiempos de menor a mayor y seleccionando el valor correspondiente al 95 % de las observaciones.

La aceleración o *speedup* se calcula comparando el tiempo sin caché con el tiempo obtenido al utilizar Redis:

$$S = \frac{T_{\text{sin caché}}}{T_{\text{con caché}}}$$

Un resultado superior a 1 indica que la utilización de caché reduce el tiempo de respuesta. Debido a que Redis no se encuentra implementado en la versión actual, no fue posible calcular un speedup real. Esta medición queda planteada para una futura versión del sistema.

Los resultados obtenidos en Postman demostraron que el backend respondió correctamente a las solicitudes de autenticación y acceso protegido. El tiempo de respuesta depende de la consulta realizada, la validación del token, la comunicación con PostgreSQL y los recursos disponibles en el equipo de ejecución.

6. Seguridad implementada

La seguridad del sistema se gestionó mediante Spring Security, autenticación JWT, control de acceso por roles y acceso a datos con Spring Data JPA. Los controles aplicados y las mejoras previstas se relacionaron con los principales riesgos descritos por OWASP Top 10.

Cuadro 10: Controles de seguridad aplicados en el sistema

Control	Implementación	Relación OWASP	Estado y evidencia
Protección de contraseñas	Las contraseñas se almacenan mediante hash BCrypt utilizando <code>PasswordEncoder</code> .	A02: Fallas criptográficas	Implementado. Puede comprobarse mediante el hash almacenado en PostgreSQL.
Autenticación JWT	El backend genera tokens firmados después de validar las credenciales del usuario.	A07: Fallas de identificación y autenticación	Implementado. Evidencia disponible en la respuesta del login de Postman.
Expiración del token	El token actual posee un tiempo de expiración configurado en el backend. La guía recomienda reducirlo a una hora.	A07: Fallas de identificación y autenticación	Implementado parcialmente. La configuración actual utiliza una duración superior a la recomendada.
Blacklist con Redis	Se propuso almacenar el identificador <code>jti</code> de los tokens revocados con un TTL equivalente a su tiempo restante.	A07: Fallas de identificación y autenticación	Diseño propuesto. Redis no se encuentra activo en la versión actual.
Configuración CORS	Spring Security establece los orígenes y métodos HTTP permitidos para la comunicación con Angular.	A05: Configuración de seguridad incorrecta	Implementado mediante las clases <code>SecurityConfig</code> y <code>WebConfig</code> .
Control de acceso por roles	Los recursos se protegen mediante reglas de seguridad y anotaciones como <code>@PreAuthorize</code> .	A01: Pérdida de control de acceso	Implementado parcialmente. La respuesta 403 Forbidden demuestra el rechazo de una operación no autorizada.
Acceso a datos sin SQL concatenado	Los repositorios extienden <code>JpaRepository</code> y utilizan parámetros administrados por Spring Data JPA.	A03: Inyección	Implementado. No se construyen consultas SQL mediante concatenación de datos del usuario.
Cabeceras HTTP de seguridad	Se consideraron las cabeceras <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> y <code>Content-Security-Policy</code> .	A05: Configuración de seguridad incorrecta	Control recomendado para completar y verificar en la configuración de Spring Security.

6.1. Evidencias de seguridad

Durante las pruebas con Postman se comprobó que una petición realizada a un endpoint protegido sin los permisos o credenciales requeridos produjo una respuesta **403 Forbidden**. Este comportamiento evidencia que Spring Security intercepta las solicitudes antes de permitir el acceso a los controladores.

Asimismo, el inicio de sesión exitoso devolvió un token JWT firmado, utilizado posteriormente mediante el esquema **Bearer**. El backend validó dicho token antes de procesar la consulta de solicitudes.



Figura 13: Acceso denegado por Spring Security a una operación protegida

6.2. Aspectos de seguridad pendientes

La versión actual proporciona una base de autenticación y autorización funcional. Sin embargo, se identificaron mejoras necesarias para cumplir completamente las recomendaciones de la entrega:

- Configurar BCrypt explícitamente con costo 12.
- Reducir la duración del access token a una hora.
- Implementar refresh token.
- Incorporar un identificador `jti` dentro del JWT.
- Implementar la revocación de tokens mediante Redis.
- Configurar y verificar las cabeceras HTTP de seguridad.
- Almacenar el access token en memoria y evitar su persistencia en `localStorage`.

La aplicación de estos controles permitirá fortalecer la protección de credenciales, el control de acceso y la administración del ciclo de vida de los tokens.

7. Docker Compose y despliegue

Docker Compose se planteó como el mecanismo de despliegue para ejecutar los componentes del sistema dentro de contenedores independientes. Esta configuración permitiría iniciar el frontend Angular, el backend Spring Boot, PostgreSQL y Redis mediante un único comando.

La arquitectura propuesta considera los siguientes servicios:

Cuadro 11: Servicios considerados en Docker Compose

Servicio	Tecnología	Puerto	Función
frontend	Angular y Nginx	4200 / 80	Presenta la interfaz web y envía solicitudes HTTP al backend.
backend	Java y Spring Boot	8080	Procesa la lógica del negocio, autenticación JWT y acceso a datos.
postgres	PostgreSQL 16	5432	Almacena usuarios, solicitudes y demás información del sistema.
redis	Redis 7	6379	Servicio propuesto para blacklist de tokens y almacenamiento temporal.

Los health checks permiten verificar que PostgreSQL y Redis se encuentren disponibles antes de iniciar el backend. De esta manera, se evita que Spring Boot intente conectarse a servicios que todavía no están preparados.

La estructura resumida del archivo de despliegue es la siguiente:

```
services:
  postgres:
    healthcheck: configurado

  redis:
    healthcheck: configurado

  backend:
    depends_on:
      postgres: healthy
      redis: healthy

  frontend:
    depends_on:
```

```
backend: started
```

En la versión actual, el sistema se ejecutó localmente mediante Angular en el puerto 4200, Spring Boot en el puerto 8080 y PostgreSQL en el puerto 5432. La incorporación completa de Docker Compose se estableció como una mejora para facilitar la reproducibilidad del entorno.

7.1. Instrucciones de ejecución

El archivo README.md debe incluir los pasos necesarios para iniciar el sistema:

1. Clonar el repositorio y acceder a la carpeta principal.
2. Configurar las variables de entorno de PostgreSQL y JWT.
3. Ejecutar el backend Spring Boot.
4. Ejecutar el frontend Angular.
5. Acceder a la aplicación desde el navegador.

Los comandos utilizados para la ejecución local son:

```
# Backend
cd backend/backend-mod
mvn spring-boot:run

# Frontend
npm install
ng serve
```

Una vez iniciados los componentes, las direcciones principales son:

Frontend:
`http://localhost:4200`

Backend:
`http://localhost:8080`

En una futura integración con Docker Compose, el sistema podrá iniciarse mediante:

```
docker compose up --build
```

8. Resultados obtenidos

El desarrollo de la presente entrega permitió disponer de una aplicación web funcional para la gestión de pre-sustentaciones de trabajos de titulación. El sistema quedó compuesto por un frontend desarrollado con Angular, un backend implementado con Spring Boot y una base de datos PostgreSQL para el almacenamiento de la información académica.

En el backend se organizaron las clases mediante paquetes de configuración, controladores, DTO, entidades, enumeraciones, repositorios, seguridad y servicios. Esta separación permitió distribuir las responsabilidades del sistema y facilitar la comprensión del código. El acceso a los datos se implementó mediante Spring Data JPA e Hibernate, evitando la construcción manual de consultas SQL para las operaciones principales.

El módulo de autenticación permitió validar las credenciales de los usuarios y generar tokens JWT. Mediante Postman se comprobó el inicio de sesión exitoso, la obtención del token y el acceso a endpoints protegidos mediante el encabezado **Authorization**. También se verificó que Spring Security rechazara solicitudes sin los permisos correspondientes mediante una respuesta **403 Forbidden**.

Además, se documentaron la arquitectura C4, el diagrama de clases, el modelo Entidad-Relación, el diccionario de datos y el flujo de autenticación. Como aspectos pendientes se identificaron la integración de Redis, refresh token, pruebas JUnit completas, Flyway y el despliegue mediante Docker Compose.

9. Conclusiones

9.1. Conclusión del primer objetivo

Se logró implementar y documentar el proceso principal de autenticación mediante Spring Security y JSON Web Token. El sistema permitió recibir las credenciales desde Angular, validar la información del usuario almacenada en PostgreSQL y generar un token JWT para acceder a los recursos protegidos. La principal dificultad técnica consistió en comprender el flujo existente entre el controlador de autenticación, el administrador de autenticación, el servicio de usuarios y el filtro JWT. Esta dificultad se resolvió mediante el análisis de las clases del paquete de seguridad y la ejecución de pruebas manuales con Postman. Las evidencias obtenidas mostraron que el endpoint de inicio de sesión devolvió un token firmado y que dicho token pudo enviarse mediante el esquema Bearer. También se comprobó que Spring Security rechazó una solicitud sin autorización mediante el código 403. Para una siguiente versión se recomienda incorporar refresh token, reducir el tiempo de expiración del access token e implementar su revocación mediante Redis.

9.2. Conclusión del segundo objetivo

La organización del backend mediante entidades, repositorios, servicios y controladores permitió mantener una separación clara de responsabilidades. Los controladores administran las solicitudes HTTP, los servicios contienen la lógica del negocio y los repositorios realizan el acceso a PostgreSQL. La principal dificultad consistió en representar la gran cantidad de clases implementadas dentro de un único diagrama UML sin afectar su legibilidad. Para resolver este problema se elaboró un modelo simplificado que incluyó únicamente las clases principales relacionadas con autenticación y gestión de solicitudes. Esta representación permitió identificar las dependencias entre `AuthController`, `SolicitudController`, `SolicitudService`, los repositorios y las entidades principales. La estructura obtenida facilita el mantenimiento y permite incorporar nuevas funcionalidades sin concentrar toda la lógica en una sola clase. En las siguientes etapas se podrá ampliar la documentación incorporando diagramas específicos por módulo.

9.3. Conclusión del tercer objetivo

Se comprobó que el sistema utiliza Spring Data JPA e Hibernate para administrar la persistencia de los usuarios, solicitudes y demás entidades del proceso académico. Los repositorios proporcionan operaciones para guardar, consultar y actualizar registros sin construir manualmente sentencias SQL mediante concatenación. La principal dificultad identificada fue la relación existente entre los usuarios y los perfiles específicos del sistema. Durante la prueba de creación de una solicitud, el backend detectó correctamente que el usuario autenticado con rol presidente no tenía un perfil de estudiante asociado. Aunque la operación no pudo completarse, la respuesta obtenida confirmó que se aplicaron las validaciones de negocio correspondientes. El acceso a PostgreSQL quedó documentado mediante el diagrama Entidad-Relación, las tablas de correspondencia Java-PostgreSQL y el diccionario de datos. Como mejora posterior se propone incorporar Flyway para controlar las migraciones y garantizar que el esquema sea reproducible.

9.4. Conclusión del cuarto objetivo

La documentación elaborada permitió reunir los elementos principales del primer módulo funcional del proyecto. Se incluyeron la arquitectura C4, el diagrama de clases, el modelo Entidad-Relación, el flujo de autenticación JWT, la estructura del repositorio, los endpoints implementados, los controles de seguridad y las pruebas ejecutadas con Postman. La dificultad principal fue diferenciar las funcionalidades que se encontraban implementadas de aquellas que correspondían a requisitos adicionales de la guía. Para evitar presentar resultados inexistentes, Redis, Docker Compose, Flyway, las pruebas automatizadas y las métricas con caché fueron descritos como ampliaciones técnicas pendientes. Las evidencias disponibles demostraron la comunicación entre Angular y Spring Boot, la generación del token JWT y la protección de los recursos. En la siguiente entrega se plantea completar las pruebas JUnit, la contenerización del sistema, la revocación de tokens y el despliegue integrado de todos los servicios.

Referencias

- [1] Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- [2] Spring Security Team. (2026). *Spring Security reference documentation*. Spring. <https://docs.spring.io/spring-security/reference/>
- [3] Spring Data Team. (2026). *Spring Data JPA reference documentation*. Spring. <https://docs.spring.io/spring-data/jpa/reference/>
- [4] Hibernate Team. (2026). *Hibernate ORM user guide*. Red Hat. https://docs.hibernate.org/orm/6.5/userguide/html_single/
- [5] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
- [6] Redgate Software. (2026). *Flyway documentation: Database migrations*. <https://documentation.red-gate.com/flyway>
- [7] Redis. (2026). *EXPIRE command documentation*. <https://redis.io/docs/latest/commands/expire/>
- [8] OWASP Foundation. (2026). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [9] OWASP Foundation. (2021). *OWASP Top 10: The ten most critical web application security risks*. <https://owasp.org/Top10/2021/>
- [10] Spring Security Team. (2026). *Cross-Origin Resource Sharing (CORS)*. Spring Security Reference. <https://docs.spring.io/spring-security/reference/servlet/integrations/cors.html>
- [11] Spring Security Team. (2026). *Security HTTP response headers*. Spring Security Reference. <https://docs.spring.io/spring-security/reference/servlet/exploits/headers.html>
- [12] Docker Inc. (2026). *Control startup and shutdown order in Docker Compose*. Docker Documentation. <https://docs.docker.com/compose/how-tos/startup-order/>
- [13] Walls, C. (2022). *Spring in Action* (6.^a ed.). Manning Publications.
- [14] Spilca, L. (2024). *Spring Security in Action* (2.^a ed.). Manning Publications.
- [15] Ahmed, S., y Mahmood, Q. (2019). An authentication based scheme for applications using JSON Web Token. En *2019 22nd International Multitopic Conference (INMIC)* (pp. 1–5). IEEE. <https://doi.org/10.1109/INMIC48123.2019.9022766>
- [16] Ethelbert, O., Moghaddam, F. F., Wieder, P., y Yahyapour, R. (2017). A JSON token-based authentication and access management schema for cloud SaaS applications. En *2017 IEEE 5th International Conference on Future Internet of Things and Cloud (FiCloud)* (pp. 47–53). IEEE. <https://doi.org/10.1109/FiCloud.2017.29>
- [17] Rahmatulloh, A., Gunawan, R., y Nursuwars, F. M. S. (2019). Performance comparison of signed algorithms on JSON Web Token. *IOP Conference Series: Materials Science and Engineering*, 550(1), 012023. <https://doi.org/10.1088/1757-899X/550/1/012023>